FastAPI & PyTorch

Interview Questions and Answers

FastAPI: Fundamentals • Routing • Pydantic • Dependency Injection • Async • Security • Middleware • Databases • WebSockets • Testing • Deployment • ML Serving

PyTorch: Tensors • Autograd • nn.Module • Losses & Optimizers • Data Loading • Training Loops • GPU & AMP • Distributed • torch.compile • Export & Quantization • Debugging • Ecosystem

Table of Contents

Part I – FastAPI

1. HTTP/REST Basics, FastAPI Fundamentals & Architecture
2. Routing, Path, Query, Header & Body Parameters
3. Pydantic Models & Data Validation
4. Responses, Status Codes & Error Handling
5. Dependency Injection
6. Async, Concurrency & Performance
7. Security: Authentication, Authorization & CORS
8. Middleware, Lifespan Events & Background Tasks
9. Databases, ORMs & Migrations
10. Project Structure, APIRouter & OpenAPI
11. WebSockets, Streaming, Files & Forms
12. Testing FastAPI Applications
13. Configuration, Deployment & Production
14. Serving ML Models with FastAPI

Part II – PyTorch

15. PyTorch Fundamentals & Tensors
16. Autograd & Computational Graphs
17. nn.Module & Building Models
18. Loss Functions, Optimizers & LR Schedulers
19. Datasets, DataLoaders & Transforms
20. Training Loop, Evaluation & Checkpointing
21. GPU, Mixed Precision & Performance
22. Distributed & Large-Scale Training
23. Advanced: Custom Autograd, Hooks, torch.compile & TorchScript
24. Model Export, Quantization, Pruning & Deployment
25. Common Architectures in PyTorch (CNN, RNN, Transformer)
26. Transfer Learning & Ecosystem
27. Debugging & Common Pitfalls
28. Coding & Scenario-Based Questions

Part I
FastAPI

1. HTTP/REST Basics, FastAPI Fundamentals & Architecture

Q1. What is an API? What is a REST API?

An **API** (Application Programming Interface) is a contract that lets one program use another program's functionality or data. A **web API** exposes it over HTTP. **REST** (Representational State Transfer) is an architectural style for web APIs with these principles:

- **Resources** are identified by URLs (`/users/42`), and you act on them with HTTP methods.
- Each request is **stateless**: it carries everything needed, such as an auth token, and the server keeps no client session between requests.
- A **client-server** split, a **uniform interface**, **cacheable** responses and a **layered system** (proxies and gateways in between).
- Representations are usually JSON.

Good REST design uses nouns in plural (`/orders`, not `/getOrders`), nests sub-resources (`/users/42/orders`), uses proper status codes, and adds versioning, filtering, pagination and consistent error bodies.

Q2. What does an HTTP request and response consist of?

Request:

- The method (`GET`, `POST`, ...)
- The URL: scheme, host, port, path and query string, e.g., `/items/5?details=true`
- Headers (`Content-Type`, `Authorization`, `Accept`, cookies)
- An optional body (JSON, form data, files)

Response:

- A status code and reason phrase (`200 OK`)
- Headers (`Content-Type`, `Set-Cookie`, `Cache-Control`)
- An optional body

In FastAPI, these parts map to path/query/header/cookie/body parameters and to `Response` objects.

Q3. What are safe and idempotent HTTP methods?

Method	Purpose	Safe	Idempotent
GET	Read a resource	Yes	Yes
HEAD / OPTIONS	Headers only / capabilities	Yes	Yes
POST	Create / trigger an action	No	No
PUT	Replace the entire resource	No	Yes
PATCH	Partial update	No	Not guaranteed
DELETE	Remove a resource	No	Yes

A **safe** method doesn't modify state. An **idempotent** method has the same effect whether it is called once or many times, so it can be retried safely. To make POST retries safe, use an `Idempotency-Key` header.

Q4. What do the HTTP status code classes mean? Name the codes you use most.

- **1xx** Informational (101 Switching Protocols for WebSockets)
- **2xx** Success: 200 OK, 201 Created, 202 Accepted (async job), 204 No Content
- **3xx** Redirection: 301/308 permanent, 302/307 temporary, 304 Not Modified (cache)
- **4xx** Client error: 400, 401, 403, 404, 405 Method Not Allowed, 409, 413 Payload Too Large, 415 Unsupported Media Type, 422, 429
- **5xx** Server error: 500, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout

FastAPI provides named constants such as `status.HTTP_201_CREATED`.

Q5. What is the difference between REST, GraphQL, gRPC and WebSockets?

REST: resource URLs plus HTTP verbs; simple and cacheable, but can over-fetch or under-fetch data. **GraphQL**: a single endpoint where the client specifies exactly which fields it wants; flexible for frontends, but caching is harder and N+1 queries are a risk. In Python, Strawberry integrates with FastAPI via `GraphQLRouter`. **gRPC**: binary Protobuf over HTTP/2 with strongly typed contracts and streaming; fast for service-to-service calls, but not browser-native. **WebSockets**: persistent bidirectional real-time channel.

Q6. How do you install FastAPI and what does `fastapi[standard]` include?

Run `pip install "fastapi[standard]"` inside a virtual environment (`python -m venv .venv`, or `uv venv`). The `standard` extra adds:

- `uvicorn[standard]` (server)
- `fastapi-cli`
- `httpx` (TestClient)
- `jinja2`
- `python-multipart` (forms and files)
- `email-validator`

Plain `pip install fastapi` installs only the core: Starlette and Pydantic.

Q7. What is a "path operation" and a "path operation function"?

In FastAPI terminology, a **path** is the URL route (`/items/{id}`) and an **operation** is the HTTP method. A **path operation decorator** (`@app.get("/items/{id}")`) binds them to a **path operation function**: the Python function that handles the request. The function's return value (dict, list, Pydantic model, str, int, None, ORM object) is automatically converted to JSON.

Q8. What is the difference between OpenAPI, Swagger and JSON Schema?

OpenAPI is the specification: a machine-readable description of an API's endpoints, parameters, schemas and auth, formerly called the Swagger Specification. **Swagger UI** is a tool that renders an OpenAPI document as interactive docs where you can try requests. **JSON Schema** describes the shape of JSON data; OpenAPI uses it for request and response bodies, which FastAPI generates from Pydantic models.

Q9. What is FastAPI and what are its key features?

FastAPI is a modern, high-performance Python web framework for building APIs, based on standard Python type hints. Key features:

- **Fast**: built on Starlette (ASGI) and Pydantic; performance comparable to Node.js/Go frameworks.
- **Type-hint driven**: request parsing, validation, serialization and docs derived from annotations.
- **Automatic interactive docs**: OpenAPI schema, Swagger UI (`/docs`) and ReDoc (`/redoc`).
- **Dependency injection** system that is simple yet powerful.
- **Native async** support (`async def`) plus sync endpoints run in a threadpool.
- Standards-based: OpenAPI, JSON Schema, OAuth2, JWT.
- Excellent editor support (autocomplete, type checks), fewer bugs.

Q10. What are Starlette and Pydantic, and what role does each play in FastAPI?

Starlette is a lightweight ASGI toolkit that provides the web layer: routing, requests/responses, middleware, WebSockets, background tasks, test client, static files. FastAPI subclasses `Starlette`. **Pydantic** provides the data layer: parsing, validation, serialization and JSON Schema generation from type-annotated models. FastAPI glues them together and adds dependency injection and OpenAPI generation.

Q11. What is ASGI and how does it differ from WSGI?

WSGI (Web Server Gateway Interface) is a synchronous standard: one request occupies one worker thread/process until done; no native WebSockets or long-lived connections (Flask, Django classic). **ASGI** (Asynchronous Server Gateway Interface) is its async successor: an app is an `async` callable `app(scope, receive, send)`; it supports HTTP, HTTP/2, WebSockets, lifespan events, and many concurrent connections on a single event loop. FastAPI is an ASGI app served by ASGI servers like Uvicorn, Hypercorn or Granian.

Q12. What is Uvicorn? How do Uvicorn and Gunicorn relate?

Uvicorn is a lightning-fast ASGI server built on `uvloop` and `httptools`. Gunicorn is a mature process manager (WSGI server). In production you can run Gunicorn with Uvicorn workers (`gunicorn -k uvicorn.workers.UvicornWorker -w 4 app:app`) to get multiple processes with supervision, or simply use `uvicorn app:app --workers 4` (or `fastapi run`). In Kubernetes, one Uvicorn process per container with replicas is often preferred.

Q13. Write a minimal FastAPI application and explain how to run it.

```
from fastapi import FastAPI

app = FastAPI(title="Demo API", version="1.0.0")

@app.get("/")
async def root():
    return {"message": "Hello World"}

@app.get("/items/{item_id}")
async def read_item(item_id: int, q: str | None = None):
    return {"item_id": item_id, "q": q}
```

Run with `uvicorn main:app --reload` (dev) or `fastapi dev main.py` / `fastapi run main.py` (FastAPI CLI). Visit `/docs` for Swagger UI.

Q14. Compare FastAPI with Flask and Django.

Aspect	FastAPI	Flask	Django / DRF
Protocol	ASGI (async-first)	WSGI (async partial)	WSGI/ASGI
Validation	Built-in via Pydantic	Extensions (Marshmallow)	DRF serializers
API docs	Automatic OpenAPI	Extensions	Extensions (drf-spectacular)
ORM / Admin	None built-in (bring your own)	None	Built-in ORM, admin, auth
Performance	Very high	Moderate	Moderate
Best for	APIs, microservices, ML serving	Small apps, flexibility	Full-stack, batteries-included apps

Q15. How does FastAPI generate API documentation automatically?

At startup FastAPI inspects every route: path, method, parameter annotations, Pydantic models for body/response, status codes, dependencies, security schemes, tags and docstrings. It builds an **OpenAPI 3.1** schema (served at `/openapi.json`) using Pydantic's JSON Schema generation. Swagger UI (`/docs`) and ReDoc (`/redoc`) render that schema. You can customize with `title`, `description`, `tags`, `summary`, `response_description`, `openapi_url=None` (disable), or override `app.openapi()`.

Q16. What happens internally when a request hits a FastAPI endpoint?

1. ASGI server (Uvicorn) parses the HTTP request and calls the app with `scope/receive/send`.
2. Middleware stack runs (outermost first): e.g., CORS, GZip, custom middleware.
3. Starlette router matches path + method to a route.
4. FastAPI resolves the dependency graph: extracts path/query/header/cookie/body params, runs dependencies (cached per request), validates with Pydantic.
5. On validation failure → `RequestValidationError` → 422 response.
6. Endpoint runs (awaited if `async def`, else in a threadpool).
7. Return value is filtered/validated against `response_model`, serialized via `jsonable_encoder`/Pydantic to JSON, wrapped in a `Response`.
8. Dependencies with `yield` run their teardown; background tasks run after the response is sent.

Q17. What are type hints and why are they central to FastAPI?

Type hints (PEP 484) annotate expected types: `def f(x: int) -> str`. Python ignores them at runtime, but FastAPI reads them via introspection to decide where a parameter comes from (path/query/body), how to convert it (string → int), how to validate it, and how to document it. `Annotated[T, metadata]` (PEP 593) lets you attach extra info like `Query(max_length=50)` or `Depends(...)` while keeping the real type.

Q18. What is the FastAPI CLI?

Installed with `pip install "fastapi[standard]"`. `fastapi dev main.py` runs Uvicorn with auto-reload on localhost for development; `fastapi run main.py` runs in production mode (listening on 0.0.0.0, no reload). It auto-detects the `app` object.

2. Routing, Path, Query, Header & Body Parameters

Q19. How does FastAPI decide whether a function parameter is a path, query or body parameter?

- If the name appears in the path template (`/items/{item_id}`) → **path parameter**.
- If it is a singular type (`int`, `str`, `float`, `bool`, `Enum`, `UUID`, `datetime`, list of these with `Query`) → **query parameter**.
- If it is a Pydantic model (or declared with `Body()`) → **request body**.
- Explicit markers override: `Path()`, `Query()`, `Header()`, `Cookie()`, `Body()`, `Form()`, `File()`.

Q20. How do you make query parameters optional, required, or with defaults? How do you add validation?

```
from typing import Annotated
from fastapi import Query

@app.get("/search")
async def search(
    q: Annotated[str, Query(min_length=3, max_length=50, pattern="^[a-z]+$")],  # required
    page: int = 1,                          # optional with default
    tag: str | None = None,                 # optional, may be None
    ids: Annotated[list[int], Query()] = [],  # ?ids=1&ids=2
):
    ...
```

A parameter with no default is required. `Query` supports `alias`, `title`, `description`, `deprecated`, `include_in_schema`, numeric constraints (`gt`, `ge`, `lt`, `le`).

Q21. Why does route declaration order matter?

Routes are matched in the order they are declared. If `/users/{user_id}` is declared before `/users/me`, the request `/users/me` matches the first route and tries to parse "me" as `user_id`. Declare fixed paths before parameterized ones.

Q22. How do you restrict a path parameter to a set of predefined values?

```
from enum import Enum
class ModelName(str, Enum):
    resnet = "resnet"
    vit = "vit"

@app.get("/models/{name}")
async def get_model(name: ModelName):
    return {"model": name.value}
```

Alternatively use `Literal["resnet", "vit"]`. Invalid values give a 422 and the options appear in docs.

Q23. How do you capture a path containing slashes (e.g., a file path)?

Use the Starlette path convertor: `@app.get("/files/{file_path:path}")`. Other convertors: `int`, `float`, `uuid`, `str`.

Q24. How do you read headers and cookies?

```
from fastapi import Header, Cookie
@app.get("/info")
async def info(user_agent: Annotated[str | None, Header()] = None,
               x_token: Annotated[list[str] | None, Header()] = None,
               session_id: Annotated[str | None, Cookie()] = None):
    ...
```

`Header` converts underscores to hyphens automatically (`user_agent` → `User-Agent`); headers are case-insensitive. Disable with `convert_underscores=False`.

Q25. How do you accept multiple body parameters or a singular value in the body?

```
@app.put("/items/{item_id}")
async def update(item_id: int, item: Item, user: User,
                 importance: Annotated[int, Body(gt=0)]):
    ...
```

With multiple body params, FastAPI expects a JSON object keyed by parameter names: `{"item": {...}, "user": {...}, "importance": 5}`. For a single model, use `Body(embed=True)` to require `{"item": {...}}` instead of the bare object.

Q26. Can you use Pydantic models for query parameters?

Yes (FastAPI ≥ 0.115): `filters: Annotated[FilterParams, Query()]` where `FilterParams` is a `BaseModel`. Set `model_config = {"extra": "forbid"}` to reject unknown query params. Same works for `Header()` and `Cookie()`.

Q27. What HTTP methods does FastAPI support and when do you use each?

`@app.get` (read), `post` (create / non-idempotent actions), `put` (full replace, idempotent), `patch` (partial update), `delete` (remove), plus `options`, `head`, `trace`. Use `@app.api_route(path, methods=[...])` for multiple. For PATCH, use `item.model_dump(exclude_unset=True)` to update only provided fields.

Q28. How do you access the raw Request object?

Declare a parameter of type `Request`: `async def f(request: Request)`. Gives access to `request.headers`, `request.client.host`, `request.url`, `await request.body()`, `await request.json()`, `request.state`, `request.app`. Data read this way bypasses validation and docs.

Q29. When should you use a path parameter vs a query parameter vs the request body?

- **Path**: identifies a specific resource, and is required (`/users/42`).
- **Query**: optional modifiers such as filtering, sorting, pagination and search (`/users?role=admin&page=2`).
- **Body**: data being created or updated, or complex or large input, sent with POST/PUT/PATCH. Avoid bodies on GET.
- **Headers**: metadata such as auth, content negotiation and correlation IDs.

Q30. How does FastAPI convert query parameter types, e.g., booleans?

Query values arrive as strings, and Pydantic converts them to the annotated type. For `bool`, the values `1`, `true`, `on` and `yes` (case-insensitive) become `True`, and `0`, `false`, `off` and `no` become `False`. Invalid values such as `?page=abc` for an `int` produce a 422 error listing the location (`["query", "page"]`) and the message.

Q31. What does a 422 validation error response look like?

```
{"detail": [{"type": "int_parsing", "loc": ["path", "item_id"],
             "msg": "Input should be a valid integer, unable to parse string as an integer",
             "input": "abc"}]}
```

`loc` shows where the error is (path, query, body field path); `type` is a machine-readable error code. You can customize the format with a `RequestValidationError` handler.

3. Pydantic Models & Data Validation

Q32. What is Pydantic and how is it used in FastAPI?

Pydantic is a data validation library that uses type hints to parse and validate data; v2 has a Rust core (`pydantic-core`) and is 5–50x faster than v1. In FastAPI, `BaseModel` subclasses define request bodies, response models, query/header models and settings. Invalid input raises `ValidationError`, which FastAPI turns into a 422 response with detailed error locations.

```
from pydantic import BaseModel, Field, EmailStr
class UserCreate(BaseModel):
    username: str = Field(min_length=3, max_length=30)
    email: EmailStr
    age: int = Field(ge=0, le=130)
    tags: list[str] = []
```

Q33. What are the key differences between Pydantic v1 and v2?

v1	v2
`.dict()`, `.json()`	`.model_dump()`, `.model_dump_json()`
`parse_obj()`	`model_validate()`, `model_validate_json()`
`class Config:`	`model_config = ConfigDict(...)`
`orm_mode = True`	`from_attributes = True`
`@validator`, `@root_validator`	`@field_validator`, `@model_validator`
`BaseSettings` in pydantic	Moved to `pydantic-settings` package
Pure Python	Rust core, much faster; strict mode; `@computed_field`; `TypeAdapter`

Q34. How do you write custom validators in Pydantic v2?

```
from pydantic import BaseModel, field_validator, model_validator

class SignUp(BaseModel):
    password: str
    confirm: str

    @field_validator("password")
    @classmethod
    def strong(cls, v: str) -> str:
        if len(v) < 8 or v.isalpha():
            raise ValueError("weak password")
        return v

    @model_validator(mode="after")
    def match(self):
        if self.password != self.confirm:
            raise ValueError("passwords do not match")
        return self
```

`mode="before"` runs before type coercion (raw input); `"after"` on validated data; `"wrap"` wraps the inner validator. Also `Annotated[str, AfterValidator(fn)]` for reusable validators.

Q35. What is `Field()` used for?

To add constraints and metadata to model fields: `default`, `default_factory`, `alias`, `title`, `description`, `examples`, `gt/ge/lt/le`, `min_length/max_length`, `pattern`, `multiple_of`, `frozen`, `exclude`. Use `default_factory=list` for mutable defaults (though Pydantic copies defaults safely anyway).

Q36. Why use separate input and output models (e.g., UserCreate, UserRead, UserInDB)?

To avoid leaking sensitive fields (password hash), to control what clients can set (no `id` / `created_at` on input), and to document each contract precisely. Common pattern: `UserBase` (shared fields) → `UserCreate(UserBase)` adds `password`; `UserRead(UserBase)` adds `id`; `UserInDB` adds `hashed_password`. Use `UserUpdate` with all-optional fields for PATCH.

Q37. What does `response_model` do?

It declares the output schema. FastAPI will: validate the return value, **filter out** fields not in the model (security), serialize it, and document it in OpenAPI. You may also use the return type annotation (`-> UserRead`). Options: `response_model_exclude_unset=True`, `response_model_exclude_none`, `response_model_include/exclude`. Set `response_model=None` to disable when returning a `Response` directly.

Q38. How do you return ORM objects through Pydantic models?

Set `model_config = ConfigDict(from_attributes=True)` on the response model (v1: `orm_mode = True`). Pydantic will read attributes (`obj.name`) instead of dict keys, so you can return a SQLAlchemy instance and FastAPI converts it via `UserRead.model_validate(db_user)`.

Q39. How do you handle nested models, lists and unions?

```
class Image(BaseModel):
    url: HttpUrl
    name: str
class Item(BaseModel):
    name: str
    images: list[Image] | None = None
    meta: dict[str, int] = {}
class Cat(BaseModel): kind: Literal["cat"]; meows: int
class Dog(BaseModel): kind: Literal["dog"]; barks: int
Pet = Annotated[Cat | Dog, Field(discriminator="kind")]
```

Discriminated unions validate faster and produce clearer errors than plain unions.

Q40. What is strict mode vs lax mode in Pydantic?

By default (lax) Pydantic coerces compatible types: `"42"` → `42`, `"true"` → `True`. Strict mode (`ConfigDict(strict=True)`, `Field(strict=True)`, or `StrictInt`) rejects mismatched types. Useful for financial or security-sensitive inputs.

Q41. What are computed fields, aliases and `extra` config?

- `@computed_field` + `@property`: derived value included in serialization and schema.
- `Field(alias="userName")`: external name differs from attribute; `populate_by_name=True` accepts both; `alias_generator=to_camel` for camelCase APIs; `serialization_alias` / `validation_alias` to differ by direction.
- `extra="ignore"` (default) / `"forbid"` (reject unknown fields) / `"allow"` (keep them).

Q42. What is `jsonable_encoder`?

A FastAPI utility that converts arbitrary objects (Pydantic models, `datetime`, `UUID`, `Decimal`, sets, ORM objects) into JSON-compatible Python types (dict, list, str, ...). Useful when storing to a NoSQL DB or building a `JSONResponse` manually.

Q43. What is `TypeAdapter`?

A Pydantic v2 tool to validate/serialize arbitrary types that are not `BaseModel`s, e.g. `TypeAdapter(list[Item]).validate_python(data)` or `.json_schema()`.

4. Responses, Status Codes & Error Handling

Q44. How do you set custom status codes?

Via the decorator: `@app.post("/items", status_code=status.HTTP_201_CREATED)`. For dynamic codes, inject `response: Response` and set `response.status_code = 202`, or return a `JSONResponse(content=..., status_code=...)`. Use 204 with no body for deletes.

Q45. What response classes are available?

- `JSONResponse` (default), `ORJSONResponse` / `UJSONResponse` (faster JSON).
- `HTMLResponse`, `PlainTextResponse`, `RedirectResponse`.
- `StreamingResponse` (generator/async generator), `FileResponse` (efficient file sending with headers).
- Generic `Response(content, media_type, headers)`.

Set app-wide default: `FastAPI(default_response_class=ORJSONResponse)`. Returning a Response directly skips `response_model` validation.

Q46. How do you raise HTTP errors?

```
from fastapi import HTTPException
@app.get("/items/{id}")
async def get(id: int):
    item = db.get(id)
    if not item:
        raise HTTPException(status_code=404, detail="Item not found",
                            headers={"X-Error": "missing"})
    return item
```

Raising (not returning) aborts the dependency chain and endpoint immediately; `detail` can be any JSON-serializable value.

Q47. How do you implement custom exception handlers?

```
class DomainError(Exception):
    def __init__(self, msg: str): self.msg = msg

@app.exception_handler(DomainError)
async def domain_handler(request: Request, exc: DomainError):
    return JSONResponse(status_code=400, content={"error": exc.msg})

@app.exception_handler(RequestValidationError)
async def validation_handler(request, exc):
    return JSONResponse(status_code=422, content={"errors": exc.errors()})
```

You can also override Starlette's `HTTPException` handler to unify the error format. Re-use defaults via `fastapi.exception_handlers.http_exception_handler`.

Q48. What is the difference between 400, 401, 403, 404, 409, 422 and 500?

400 Bad Request (malformed / business-rule violation); 401 Unauthorized (missing/invalid credentials — include `WWW-Authenticate`); 403 Forbidden (authenticated but not allowed); 404 Not Found; 409 Conflict (duplicate resource / version conflict); 422 Unprocessable Entity (FastAPI's validation errors); 429 Too Many Requests; 500 Internal Server Error (unhandled exception); 503 Service Unavailable (dependency down / overloaded).

Q49. How do you document multiple possible responses?

```
@app.get("/items/{id}", response_model=Item,
         responses={404: {"model": ErrorMsg, "description": "Not found"},
                    200: {"content": {"image/png": {}}}})
```

Q50. How do you set headers and cookies on a response?

Inject `response: Response` and call `response.headers["X-Cache"] = "HIT"` or `response.set_cookie("session", token, httponly=True, secure=True, samesite="lax")`; or set them on a returned `JSONResponse`.

5. Dependency Injection

Q51. What is dependency injection in FastAPI and why is it useful?

A dependency is any callable (function, class, async function, generator) declared via `Depends()`. FastAPI calls it, resolving its own parameters (path, query, other dependencies) recursively, and injects the result. Benefits: code reuse (pagination, auth, DB session), separation of concerns, easy testing via overrides, automatic documentation of the dependency's parameters, and resource lifecycle management with `yield`.

```
async def pagination(skip: int = 0, limit: int = Query(10, le=100)):
    return {"skip": skip, "limit": limit}
Pagination = Annotated[dict, Depends(pagination)]

@app.get("/items")
async def items(p: Pagination): ...
```

Q52. Explain dependencies with `yield`.

Code before `yield` is setup; the yielded value is injected; code after `yield` runs as teardown after the response is produced (similar to a context manager). Typical for DB sessions:

```
def get_db():
    db = SessionLocal()
    try:
        yield db
        db.commit()
    except Exception:
        db.rollback()
        raise
    finally:
        db.close()
```

Since FastAPI 0.106, exit code runs before the response is sent (so you cannot use the resource in background tasks; create a new session there). Exceptions raised in the endpoint propagate into the dependency's `except` block.

Q53. What are sub-dependencies and dependency caching?

Dependencies can depend on other dependencies, forming a DAG. Within a single request, if the same dependency is used multiple times, it is executed **once** and the result cached. Disable with `Depends(dep, use_cache=False)`.

Q54. How do you use classes as dependencies?

Any callable works; a class's `__init__` parameters become request parameters: `commons: Annotated[CommonParams, Depends()]` (shortcut: empty `Depends()` uses the annotated type). A **callable instance** (class with `__call__`) allows parameterized dependencies, e.g., `Depends(RoleChecker(["admin"]))`.

Q55. How do you apply dependencies to a whole router or app without using their return values?

`@app.get("/x", dependencies=[Depends(verify_key)])`, `APIRouter(dependencies=[Depends(get_current_user)])`, or `FastAPI(dependencies=[Depends(log_request)])`. Useful for auth checks, rate limiting, logging.

Q56. How do you override dependencies in tests?

```
def fake_user(): return User(id=1, name="test")
app.dependency_overrides[get_current_user] = fake_user
# ... run tests ...
app.dependency_overrides.clear()
```

Q57. What is `Security()` vs `Depends()`?

`Security` is a subclass of `Depends` that additionally accepts `scopes=[...]`. Scopes are collected into `SecurityScopes` for OAuth2 scope-based authorization and documented in OpenAPI.

Q58. Should dependencies be sync or async?

Either; FastAPI awaits async ones and runs sync ones in the threadpool. Make it `async` if it awaits I/O (async DB, HTTP client); keep it sync if it does blocking I/O or is trivial CPU work. Mixing is fine.

6. Async, Concurrency & Performance

Q59. What are coroutines, the event loop and `async` / `await`?

An `async def` function returns a **coroutine** object when called; it doesn't run until awaited or scheduled. `await` suspends the coroutine until the awaited operation (e.g., a network response) finishes, which gives control back to the **event loop**. The event loop is a single-threaded scheduler that runs whichever task is ready. While one request waits on the database, the loop serves other requests, so one process can handle thousands of concurrent I/O-bound connections.

```
async def fetch_user(id):
    await asyncio.sleep(1)        # non-blocking: other tasks run meanwhile
    return {"id": id}

async def main():
    users = await asyncio.gather(*(fetch_user(i) for i in range(100)))  # ~1 s, not 100 s
```

You can only `await` inside `async def`. Forgetting `await` returns an un-run coroutine (Python warns: "coroutine was never awaited").

Q60. Threads vs processes vs asyncio: when to use each?

	asyncio	Threads	Processes
Best for	Many concurrent I/O waits	Blocking I/O libraries	CPU-bound work
Parallel CPU	No	No (GIL)	Yes
Overhead	Very low	Medium	High (memory, IPC)
Switching	Cooperative (at `await`)	Preemptive by OS	Separate interpreters

Q61. When should you use `async def` vs `def` for path operations?

- `async def`: when calling libraries that support `await` (httpx.AsyncClient, asyncpg, SQLAlchemy async, aioredis, motor). Runs directly on the event loop.
- `def`: when using blocking libraries (requests, psycopg2, sync SQLAlchemy, file I/O, most ML inference). FastAPI runs it in an external threadpool (AnyIO, default 40 threads) so it does not block the loop.
- **Never** perform blocking calls (`time.sleep`, `requests.get`, heavy CPU) inside `async def` — this freezes the whole event loop and all concurrent requests.

Q62. Explain concurrency vs parallelism in the context of FastAPI.

Concurrency: many tasks make progress by interleaving while waiting on I/O (event loop, async/await, single thread). **Parallelism**: tasks run simultaneously on multiple CPU cores (multiple processes). FastAPI achieves concurrency with asyncio and threads; for CPU parallelism, run multiple worker processes (Uvicorn/Gunicorn workers) or offload to a process pool / task queue, because of Python's GIL.

Q63. How do you run CPU-bound or blocking code from an async endpoint?

```
from fastapi.concurrency import run_in_threadpool
result = await run_in_threadpool(blocking_fn, arg)          # I/O-bound blocking

loop = asyncio.get_running_loop()
result = await loop.run_in_executor(process_pool, cpu_fn, x)  # CPU-bound
```

For long jobs, use a task queue (Celery, RQ, Arq, Dramatiq) and return a job ID (202 Accepted) with a status endpoint.

Q64. How do you make concurrent outbound HTTP calls?

```
async with httpx.AsyncClient(timeout=5) as client:
    r1, r2 = await asyncio.gather(client.get(url1), client.get(url2))
```

Reuse a single client (created in lifespan) for connection pooling instead of creating one per request.

Q65. What techniques improve FastAPI performance?

- Use async drivers end-to-end; avoid blocking the loop.
- Multiple workers / replicas; tune threadpool size for sync endpoints.
- `ORJSONResponse`; return Pydantic models directly; avoid double validation.
- Connection pooling (DB, HTTP, Redis); create clients in lifespan.
- Caching (Redis, `fastapi-cache`, HTTP cache headers, ETags); GZip middleware for large responses.
- Pagination, selecting only needed columns, avoiding N+1 queries.
- `uvloop` + `httptools` (included in `uvicorn[standard]`).
- Profile with py-spy, pyinstrument; load test with Locust/k6.

Q66. What is the GIL and how does it affect FastAPI apps?

The Global Interpreter Lock lets only one thread execute Python bytecode at a time per process. I/O-bound work is fine (the GIL is released during I/O), but CPU-bound Python code in threads does not run in parallel. Solutions: multiple processes, native libraries that release the GIL (NumPy, PyTorch), process pools, or free-threaded Python 3.13+ builds.

Q67. How would you implement rate limiting?

Use `slowapi` (limits-based, decorator per route) or a custom middleware/dependency using Redis (fixed window with `INCR` + `EXPIRE`, sliding window with sorted sets, or token bucket). Key on API key / user ID / IP; return 429 with `Retry-After`. In production often done at the API gateway (NGINX, Kong, Azure APIM).

7. Security: Authentication, Authorization & CORS

Q68. What security utilities does FastAPI provide?

`fastapi.security` includes `OAuth2PasswordBearer`, `OAuth2PasswordRequestForm`, `OAuth2AuthorizationCodeBearer`, `HTTPBasic`, `HTTPBearer`, `APIKeyHeader`, `APIKeyQuery`, `APIKeyCookie`, `OpenIdConnect`, `SecurityScopes`. They extract credentials and register security schemes in OpenAPI (the "Authorize" button in Swagger).

Q69. Implement JWT authentication with OAuth2 password flow.

```
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
pwd = PasswordHash.recommended()          # pwdlib (argon2); or passlib bcrypt

@app.post("/token")
async def login(form: Annotated[OAuth2PasswordRequestForm, Depends()]):
    user = authenticate(form.username, form.password)
    if not user:
        raise HTTPException(401, "Incorrect credentials",
                            headers={"WWW-Authenticate": "Bearer"})
    exp = datetime.now(timezone.utc) + timedelta(minutes=30)
    token = jwt.encode({"sub": user.username, "exp": exp}, SECRET, algorithm="HS256")
    return {"access_token": token, "token_type": "bearer"}

async def get_current_user(token: Annotated[str, Depends(oauth2_scheme)]):
    try:
        payload = jwt.decode(token, SECRET, algorithms=["HS256"])
    except InvalidTokenError:
        raise HTTPException(401, "Invalid token", headers={"WWW-Authenticate": "Bearer"})
    user = get_user(payload.get("sub"))
    if user is None: raise HTTPException(401)
    return user
```

Keep the secret in env/secret store, use short-lived access tokens + refresh tokens, always pin the allowed `algorithms`, and hash passwords with argon2/bcrypt (never store plaintext).

Q70. Authentication vs authorization? How do you implement role-based access control?

Authentication verifies *who* you are; authorization decides *what* you may do. RBAC via a parameterized dependency:

```
class RequireRole:
    def __init__(self, *roles): self.roles = roles
    def __call__(self, user: User = Depends(get_current_user)):
        if user.role not in self.roles:
            raise HTTPException(403, "Forbidden")
        return user

@app.delete("/users/{id}", dependencies=[Depends(RequireRole("admin"))])
```

Alternatively OAuth2 scopes with `Security(get_current_user, scopes=["items:write"])`. Also enforce object-level checks (user owns the resource) to prevent IDOR.

Q71. What is an "origin"? Which of these URLs share an origin?

An origin is the triple **scheme + host + port**. Path, query and fragment don't matter.

Compared with `https://app.example.com`	Same origin?	Reason
`https://app.example.com/api/users`	Yes	Only the path differs
`http://app.example.com`	No	Different scheme
`https://api.example.com`	No	Different host (subdomains count)
`https://app.example.com:8443`	No	Different port (default 443)

Common gotcha: `http://localhost:3000` and `http://127.0.0.1:3000` are **different** origins.

Q72. What is the Same-Origin Policy (SOP)?

A browser security rule: scripts on a page from one origin can't **read** responses from another origin. It stops a malicious site from using your logged-in session (cookies) to read your bank or email data. The SOP allows embedding cross-origin resources (`<img>`, `<script>`, `<link>`) and submitting forms, but it blocks JavaScript (`fetch`/XHR) from reading the responses. CORS is how a server deliberately **relaxes** the SOP for chosen origins.

Q73. What is CORS and why do you need it?

Cross-Origin Resource Sharing is an HTTP-header-based protocol. A server uses it to tell the browser which other origins may read its responses. It's needed whenever the frontend and API are on different origins, e.g., a React app on `https://app.example.com` or `http://localhost:5173` calling a FastAPI backend on `https://api.example.com` or `http://localhost:8000`. The browser sends an `Origin` header. If the response lacks a matching `Access-Control-Allow-Origin` header, the browser hides the response from JavaScript and reports a CORS error.

Q74. Is CORS enforced by the server or the browser? Does CORS protect your API?

The **browser** enforces it. The server only sends headers. So:

- curl, Postman, Python `requests` and server-to-server calls ignore CORS completely. CORS is **not** authentication or access control, so always protect endpoints with auth.
- For simple requests, the server still **runs** the request. The browser only hides the response from JavaScript, so side effects still happen.
- CORS protects **users** (their cookies and data in the browser), not your server.

Q75. What are "simple" requests and "preflight" requests?

A **simple request** is sent directly with an `Origin` header. It must meet all of these conditions:

- The method is `GET`, `HEAD` or `POST`.
- Only CORS-safelisted headers are used (`Accept`, `Accept-Language`, `Content-Language`, `Content-Type`).
- The `Content-Type` is `text/plain`, `multipart/form-data` or `application/x-www-form-urlencoded`.

Anything else triggers a **preflight**: before the real request, the browser sends an `OPTIONS` request with `Origin`, `Access-Control-Request-Method` and `Access-Control-Request-Headers`. The real request is sent only if the server's reply allows it. In practice most API calls are preflighted, because `Content-Type: application/json`, an `Authorization` header, `PUT` / `PATCH` / `DELETE`, or any custom `X-*` header triggers one.

```
OPTIONS /items HTTP/1.1                     HTTP/1.1 200 OK
Origin: https://app.example.com             Access-Control-Allow-Origin: https://app.example.com
Access-Control-Request-Method: POST         Access-Control-Allow-Methods: GET, POST
Access-Control-Request-Headers: content-type, authorization
                                            Access-Control-Allow-Headers: content-type,
authorization
                                            Access-Control-Max-Age: 600
                                            Vary: Origin
```

Q76. Explain the main CORS response headers.

Header	Meaning
`Access-Control-Allow-Origin`	The single allowed origin, or `*`. It can't be a list.
`Access-Control-Allow-Methods`	Methods allowed for the actual request (preflight reply)
`Access-Control-Allow-Headers`	Request headers the client may send (preflight reply)
`Access-Control-Allow-Credentials`	`true` lets the browser send cookies or auth and expose the response
`Access-Control-Expose-Headers`	Response headers JavaScript may read beyond the safelisted ones (e.g., `X-Total-Count`, `Content-Disposition`)
`Access-Control-Max-Age`	How many seconds the browser may cache the preflight result. Browsers cap it, e.g., Chrome at 2 hours.
`Vary: Origin`	Tells caches/CDNs that the response differs per Origin, so they don't serve one origin's headers to another

Q77. How do you configure CORS in FastAPI? Explain each `CORSMiddleware` parameter.

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://app.example.com", "http://localhost:5173"],
    allow_origin_regex=r"https://.*\.example\.com",   # optional
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE"],
    allow_headers=["Authorization", "Content-Type"],
    expose_headers=["X-Total-Count"],
    max_age=600,
)
```

- `allow_origins`: exact origins, with no trailing slash or path. Default `[]`.
- `allow_origin_regex`: regex that is full-matched against the origin, useful for subdomains or preview deployments. Escape dots, otherwise `.` matches any character.
- `allow_methods`: default `["GET"]`.
- `allow_headers`: default `[]`. Safelisted headers are always allowed.
- `allow_credentials`: default `False`.
- `expose_headers`: default `[]`.
- `max_age`: preflight cache time in seconds. Default 600.

The middleware answers preflight `OPTIONS` requests itself; you don't need `@app.options` routes. It applies to the whole app, not per router. For different policies, mount a sub-application with its own middleware.

Q78. Why can't you use `allow_origins=["*"]` with `allow_credentials=True`?

By the CORS spec, browsers reject `Access-Control-Allow-Origin: *` on credentialed requests (cookies, HTTP auth, client certs). The FastAPI docs say that when credentials are allowed, origins, methods and headers must all be listed explicitly. Some Starlette versions handle this combination by reflecting whatever `Origin` the request sent. That effectively lets **every website** make authenticated requests as your users, which is a serious vulnerability. The same risk comes from code that blindly echoes the `Origin` header, or from a loose regex such as `.*example.com`, which also matches `evilexample.com`.

Q79. What does the frontend need to do to send cookies cross-origin?

- Client: `fetch(url, { credentials: "include" })` or axios `withCredentials: true`.
- Server: `allow_credentials=True` and an explicit origin.
- Cookie: `SameSite=None; Secure` (HTTPS required) for cross-*site* requests. With `Lax` / `Strict`, browsers won't send the cookie on cross-site fetches.
- Bearer tokens in the `Authorization` header don't need `credentials: "include"`, but the `Authorization` header must be in `allow_headers`.

Q80. What is the difference between CORS and CSRF? Does CORS prevent CSRF?

CSRF (Cross-Site Request Forgery) tricks a logged-in user's browser into **sending** a state-changing request, such as an auto-submitted form POST, that carries their cookies. **CORS** only controls whether a cross-origin script can **read** the response. Simple requests such as form POSTs aren't preflighted, so CORS does **not** prevent CSRF. Defenses against CSRF:

- `SameSite` cookies
- CSRF tokens (double-submit cookie)
- Requiring a custom header or JSON content type, which forces a preflight
- Checking `Origin` / `Referer` on unsafe methods
- Using bearer tokens instead of cookies

Q81. The browser shows a CORS error but the endpoint works in Postman. How do you debug it?

1. Open DevTools → Network. Check whether the **preflight** (`OPTIONS`) or the actual request failed, and inspect the request `Origin` and the response headers.
2. Make sure the origin exactly matches `allow_origins`: scheme, port, no trailing slash, `localhost` vs `127.0.0.1`.
3. Make sure the requested method and headers (e.g., `Authorization`, `Content-Type`, custom `X-*`) are allowed. Starlette returns **400 "Disallowed CORS ..."** to the preflight otherwise.
4. Check whether the real error is a **500 or 404**. If an unhandled exception occurs, Starlette's `ServerErrorMiddleware` (outermost) returns a 500 **without** CORS headers, so the browser reports a misleading CORS error. Check the server logs.
5. Check for redirects (e.g., a trailing-slash 307 redirect to `http://` behind a proxy). Preflights can't follow redirects.
6. Check whether a reverse proxy, API gateway or CDN strips or duplicates CORS headers. Two `Access-Control-Allow-Origin` values are also invalid.
7. For credentials: check that the origin isn't `*`, that `allow_credentials=True` is set, and that the cookie has `SameSite=None; Secure`.

Q82. Does the order of middleware matter for CORS?

Yes. In Starlette, the middleware added **last** runs **first** (outermost). `CORSMiddleware` should wrap everything that can reject a request, such as auth, rate-limiting or custom middleware returning 401/429. Otherwise those responses lack CORS headers, and the frontend can't read the error status. It also has to answer the preflight before auth middleware rejects it: preflights never carry credentials, so auth checks shouldn't apply to `OPTIONS`. Add `CORSMiddleware` last, or handle auth in dependencies, which run after the middleware.

Q83. Do CORS rules apply to WebSockets?

No. Browsers send an `Origin` header during the WebSocket handshake but don't enforce CORS, and `CORSMiddleware` doesn't affect WebSocket routes. Validate `websocket.headers.get("origin")` against an allow-list yourself, and close with code 1008 if it isn't allowed. This prevents Cross-Site WebSocket Hijacking, especially with cookie auth.

Q84. How can you avoid CORS entirely?

Serve the frontend and API from the **same origin**:

- A reverse proxy (NGINX, Traefik, Azure Front Door) routes `/` to the SPA and `/api` to FastAPI.
- Or FastAPI serves the built SPA via `StaticFiles`.
- In development, use the frontend dev-server proxy (Vite `server.proxy`, CRA `proxy`).

This removes preflight latency and simplifies cookie auth.

Q85. How do you manage CORS origins per environment and test them?

```
class Settings(BaseSettings):
    cors_origins: list[str] = []          # env: CORS_ORIGINS='["https://app.example.com"]'

app.add_middleware(CORSMiddleware, allow_origins=settings.cors_origins, ...)

def test_preflight_allowed():
    r = client.options("/items", headers={
        "Origin": "https://app.example.com",
        "Access-Control-Request-Method": "POST",
        "Access-Control-Request-Headers": "content-type"})
    assert r.status_code == 200
    assert r.headers["access-control-allow-origin"] == "https://app.example.com"

def test_preflight_blocked():
    r = client.options("/items", headers={"Origin": "https://evil.com",
                                          "Access-Control-Request-Method": "POST"})
    assert r.status_code == 400
```

Allow `localhost` origins only in development, and never allow the `"null"` origin, which sandboxed iframes and `file://` pages send.

Q86. How do you implement API key authentication?

```
api_key_header = APIKeyHeader(name="X-API-Key", auto_error=False)
async def verify_key(key: str | None = Security(api_key_header)):
    if key is None or not secrets.compare_digest(key, settings.api_key):
        raise HTTPException(401, "Invalid API key")
```

Use `secrets.compare_digest` to avoid timing attacks; store hashed keys in DB for multi-tenant systems.

Q87. What security best practices apply to FastAPI services?

- HTTPS everywhere (TLS at proxy), HSTS; `TrustedHostMiddleware`, `HTTPSRedirectMiddleware`.
- Validate all input with Pydantic (lengths, patterns, `extra="forbid"`); use ORM parameterized queries (prevent SQL injection).
- Use `response_model` to avoid leaking sensitive fields.
- Secrets from env / Key Vault; never commit them. Rotate keys.
- Rate limiting, request size limits, timeouts.
- Disable docs in production if not needed (`docs_url=None`).
- Don't leak stack traces; generic 500 handler with logging.
- Secure cookies (`HttpOnly`, `Secure`, `SameSite`); CSRF protection for cookie-based auth.
- Dependency scanning (pip-audit), least-privilege DB users.

Q88. How do you integrate with an external identity provider (Azure AD / Entra ID, Auth0, Keycloak)?

Use OAuth2 Authorization Code (+PKCE) flow for users; the API acts as a resource server validating bearer JWTs: fetch the provider's JWKS (cache it), verify signature (RS256), `iss`, `aud`, `exp`, `nbf`, and scopes/roles claims. Libraries: `fastapi-azure-auth`, `authlib`, `PyJWT` with `PyJWKClient`.

Q89. Session-based vs token-based authentication?

Session: after login, the server stores session state (in a DB or Redis) and sends a session-ID cookie. Revocation is easy, but the server is stateful, sticky sessions or a shared store are needed, and you need CSRF protection. **Token (JWT)**: the server issues a signed, self-contained token that the client sends in `Authorization: Bearer`. It is stateless and scales across services, but is hard to revoke before it expires and makes payloads larger. Many apps use short-lived JWT access tokens plus refresh tokens stored in HttpOnly cookies.

Q90. What is the structure of a JWT? Is its payload encrypted?

A JWT is three Base64URL-encoded parts joined by dots: `header.payload.signature`.

- **Header**: algorithm and type, e.g., `{"alg": "HS256", "typ": "JWT"}`.
- **Payload**: claims such as `sub`, `exp`, `iat`, `iss`, `aud`, roles and scopes.
- **Signature**: HMAC (HS256, shared secret) or RSA/ECDSA (RS256/ES256, private key signs and public key verifies).

The payload is only **encoded**, not encrypted; anyone can read it, so never put secrets in it (use JWE if you need encryption). The signature guarantees integrity. Always verify the signature, pin the allowed algorithms (reject `alg: none`), and check `exp`, `aud` and `iss`.

Q91. How do you implement refresh tokens, logout and token revocation with JWTs?

- Issue a short-lived access token (5–15 minutes) and a long-lived refresh token (days). Store the refresh token in an HttpOnly, Secure cookie or in the DB.
- `POST /refresh` validates the refresh token and issues new tokens. With **rotation**, each refresh token is single-use; if a used one is presented again, revoke the whole token family because it was likely stolen.
- **Logout/revocation**: delete the refresh token server-side; keep a deny-list of access-token `jti` values in Redis until they expire; or include a `token_version` claim that you compare with the user record.

Q92. Name the OAuth2 flows (grant types) and when each is used.

- **Authorization Code + PKCE**: web, SPA and mobile apps where users log in with an identity provider. This is the recommended default.
- **Client Credentials**: machine-to-machine calls with no user.
- **Device Code**: TVs and CLIs with limited input.
- **Refresh Token**: obtain new access tokens.
- **Password (ROPC)**: the app collects the username and password directly. It is legacy, discouraged except for first-party or simple setups, and is what FastAPI's tutorial uses for simplicity.
- **Implicit**: deprecated.

OpenID Connect adds an identity layer on top of OAuth2: an ID token plus a userinfo endpoint.

Q93. Why hash passwords with bcrypt/argon2 instead of SHA-256?

General-purpose hashes like SHA-256 are designed to be fast, which lets attackers try billions of guesses per second on GPUs. Password hashing algorithms (argon2id, bcrypt, scrypt) are deliberately slow, include a per-password **salt** that defeats rainbow tables, and argon2 is also memory-hard. Never encrypt passwords reversibly. Compare hashes with a constant-time check (the library does this), and re-hash on login when you change parameters.

8. Middleware, Lifespan Events & Background Tasks

Q94. What is middleware and how do you write one?

Middleware wraps every request/response, running code before the endpoint and after the response is generated.

```
@app.middleware("http")
async def add_timing(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    response.headers["X-Process-Time"] = f"{time.perf_counter() - start:.4f}"
    return response
```

Class-based: subclass `BaseHTTPMiddleware` or write a pure ASGI middleware (faster, works with streaming). Middleware added last runs first (outermost). Built-ins: `CORSMiddleware`, `GZipMiddleware`, `TrustedHostMiddleware`, `HTTPSRedirectMiddleware`, `SessionMiddleware`.

Q95. Middleware vs dependencies — when to use which?

Middleware: cross-cutting concerns for *all* requests at the HTTP level (logging, timing, CORS, compression, request IDs, security headers); no access to parsed params, not in OpenAPI. **Dependencies**: per-route/router logic needing typed inputs (auth, DB session, pagination), documented, testable via overrides, can return values to endpoints.

Q96. What are lifespan events and how do you use them?

Code executed once at startup and shutdown — e.g., load an ML model, create DB/HTTP connection pools, warm caches, close resources.

```
from contextlib import asynccontextmanager
@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.model = load_model()
    app.state.http = httpx.AsyncClient()
    yield
    await app.state.http.aclose()

app = FastAPI(lifespan=lifespan)
```

`@app.on_event("startup"/"shutdown")` is deprecated in favor of `lifespan`. Note: lifespan runs per worker process.

Q97. What are BackgroundTasks? When should you NOT use them?

```
def send_email(to: str, body: str): ...
@app.post("/signup")
async def signup(user: UserCreate, bg: BackgroundTasks):
    bg.add_task(send_email, user.email, "Welcome!")
    return {"ok": True}
```

They run in the same process after the response is sent — good for small fire-and-forget tasks (emails, logging, cache invalidation). Not suitable for heavy/long jobs, jobs needing retries, persistence, scheduling or distribution across machines — tasks are lost if the process dies. Use Celery/RQ/Arq/Azure Queue + worker instead.

Q98. How would you add a correlation/request ID to all logs?

Middleware reads `X-Request-ID` or generates a UUID, stores it in a `contextvars.ContextVar` (async-safe), adds it to the response header, and a logging filter injects it into each log record. Libraries: `asgi-correlation-id`. Combine with structured JSON logging and OpenTelemetry tracing.

Q99. How do you write a pure ASGI middleware, and why prefer it over `BaseHTTPMiddleware`?

```
class SecurityHeadersMiddleware:
    def __init__(self, app): self.app = app
    async def __call__(self, scope, receive, send):
        if scope["type"] != "http":
            return await self.app(scope, receive, send)
        async def send_wrapper(message):
            if message["type"] == "http.response.start":
                message.setdefault("headers", []).append((b"x-content-type-options", b"nosniff"))
            await send(message)
        await self.app(scope, receive, send_wrapper)

app.add_middleware(SecurityHeadersMiddleware)
```

`BaseHTTPMiddleware` is easy to write but has overhead, can interfere with streaming responses and background tasks, and has had issues propagating `contextvars`. A pure ASGI middleware works directly on the `scope/receive/send` messages: it is faster, supports streaming and WebSockets, and is the right choice for libraries.

Q100. How do you integrate Celery (or another task queue) with FastAPI?

```
# tasks.py
celery = Celery("app", broker="redis://redis:6379/0", backend="redis://redis:6379/1")
@celery.task(bind=True, max_retries=3)
def generate_report(self, user_id): ...

# api
@app.post("/reports", status_code=202)
def create_report(user: CurrentUser):
    task = generate_report.delay(user.id)
    return {"task_id": task.id}

@app.get("/reports/{task_id}")
def status(task_id: str):
    r = AsyncResult(task_id, app=celery)
    return {"state": r.state, "result": r.result if r.ready() else None}
```

Workers run as separate processes or containers and scale independently. Pass IDs rather than large objects, make tasks idempotent, and configure retries and time limits. Async-native alternatives include Arq, Dramatiq, Taskiq and Azure Service Bus with Functions.

Q101. How can you log or modify the raw request body for every route?

Middleware that reads the body consumes the stream. Instead, use a custom `APIRoute` subclass that overrides `get_route_handler()`: it reads `await request.body()` (which caches it), logs or decompresses it, and calls the original handler. Then set `APIRouter(route_class=LoggingRoute)`. The same technique lets you catch exceptions per route with access to the body.

9. Databases, ORMs & Migrations

Q102. How do you integrate SQLAlchemy with FastAPI?

```
engine = create_async_engine("postgresql+asyncpg://...", pool_size=10, pool_pre_ping=True)
SessionLocal = async_sessionmaker(engine, expire_on_commit=False)

async def get_session():
    async with SessionLocal() as session:
        yield session
SessionDep = Annotated[AsyncSession, Depends(get_session)]

@app.get("/users/{id}", response_model=UserRead)
async def get_user(id: int, db: SessionDep):
    user = await db.get(User, id)
    if not user: raise HTTPException(404)
    return user
```

One session per request via a `yield` dependency; engine/pool created once per process.

Q103. What is SQLModel?

A library by FastAPI's author combining SQLAlchemy and Pydantic: one class can be both a table model (`table=True`) and a Pydantic model, reducing duplication. Still recommended to have separate Create/Read/Update models for API contracts.

Q104. Sync vs async database access in FastAPI?

Sync drivers (psycopg2, sync SQLAlchemy) should be used in `def` endpoints (threadpool). Async drivers (asyncpg, aiomysql, aiosqlite, motor for MongoDB) with `async def`. Async shines with many concurrent I/O-bound requests; beware lazy loading in async SQLAlchemy (use `selectinload` / `joinedload`, since implicit IO raises errors).

Q105. How do you manage schema migrations?

Use **Alembic**: `alembic init`, point `env.py` to your `Base.metadata`, `alembic revision --autogenerate -m "msg"`, review, `alembic upgrade head`. Run migrations as a separate deployment step/job, not at app startup with multiple workers (race conditions).

Q106. What is the N+1 query problem and how do you avoid it?

Fetching N parent rows and then issuing one query per row for related data (N+1 queries). Avoid with eager loading (`selectinload`, `joinedload`), joins, or batching (DataLoader pattern in GraphQL).

Q107. How do you implement pagination?

Offset/limit: simple, supports jumping to pages, but slow for large offsets and inconsistent under inserts. **Cursor/keyset**: `WHERE id > :last_id ORDER BY id LIMIT n`; stable and fast for large datasets. Return metadata (`total`, `next_cursor`). Library: `fastapi-pagination`.

Q108. How do you handle transactions and concurrency conflicts?

Use `async with session.begin():` for atomic units; roll back on exceptions. Handle concurrent updates with optimistic locking (version column; 409 on mismatch) or pessimistic locking (`SELECT ... FOR UPDATE`). Make POST endpoints idempotent with idempotency keys where needed.

Q109. How do you use Redis or MongoDB with FastAPI?

Create the client in lifespan (`redis.asyncio.Redis`, `motor.AsyncIOMotorClient` / `pymongo` async), store on `app.state`, and expose via dependency. Redis: caching, rate limiting, sessions, pub/sub, queues. MongoDB: use Pydantic models for validation; Beanie ODM integrates natively.

Q110. How do you prevent SQL injection in FastAPI apps?

Never build SQL with string formatting from user input (`f"SELECT * FROM users WHERE name = '{name}'"`). Use ORM queries (`select(User).where(User.name == name)`) or bound parameters (`text("... WHERE name = :name").bindparams(name=name)`), which send data separately from the SQL. Validate inputs with Pydantic. For dynamic sort columns, map them to an allow-list, because identifiers can't be bound as parameters. Connect with a least-privilege DB user.

Q111. What caching strategies would you use in a FastAPI service?

- **Cache-aside**: check Redis first; on a miss, query the DB, store the result with a TTL, and invalidate it on writes.
- **HTTP caching**: `Cache-Control`, `ETag` / `If-None-Match` → 304 responses, CDN caching for public GET responses.
- **In-process**: `functools.lru_cache` or `cachetools.TTLCache` for config and small lookups. Note that each worker has its own copy.

Watch for stale data, cache stampedes (use locks or request coalescing), and leaking private data through shared caches. `fastapi-cache2` provides decorators.

10. Project Structure, APIRouter & OpenAPI

Q112. How do you structure a large FastAPI application?

```
app/
  main.py            # create app, include routers, middleware, lifespan
  core/config.py     # pydantic-settings
  core/security.py
  api/deps.py        # shared dependencies
  api/v1/routes/users.py, items.py
  models/            # SQLAlchemy ORM models
  schemas/           # Pydantic request/response models
  services/          # business logic
  repositories/      # data access
  db/session.py
tests/
alembic/
```

Keep routes thin; business logic in services; data access in repositories; organize by feature/domain for big projects.

Q113. What is `APIRouter`?

```
router = APIRouter(prefix="/users", tags=["users"],
                   dependencies=[Depends(get_current_user)],
                   responses={404: {"description": "Not found"}})
@router.get("/{id}")
async def get_user(id: int): ...

# main.py
app.include_router(router, prefix="/api/v1")
```

Routers group related endpoints with shared prefix, tags, dependencies and responses; they can be nested (`router.include_router(...)`).

Q114. How do you version an API?

URL path versioning (`/api/v1`, `/api/v2`) via separate routers (most common); header versioning (`Accept: application/vnd.api.v2+json`); or mounting separate sub-apps per version. Mark old endpoints `deprecated=True` to flag them in docs.

Q115. What is mounting a sub-application?

`app.mount("/v2", v2_app)` attaches an independent ASGI app (its own docs, middleware). Also used for static files (`app.mount("/static", StaticFiles(directory="static"))`) or mounting Flask/Django via WSGIMiddleware, or Gradio apps.

Q116. How do you customize the OpenAPI schema and docs?

- `FastAPI(title, description, version, openapi_tags=[...], servers=[...], contact, license_info)`.
- Per-route: `summary`, `description` (or docstring), `tags`, `operation_id`, `include_in_schema=False`, `deprecated`, `openapi_extra`.
- Examples: `Field(examples=[...])`, `model_config = {"json_schema_extra": {...}}`, `Body(openapi_examples=...)`.
- Override `app.openapi` using `get_openapi()` for full control; change `docs_url`, `redoc_url`, `openapi_url`.
- Generate typed clients from the schema with openapi-generator / openapi-ts; set `generate_unique_id_function` for clean names.

Q117. How do you create a generic paginated response model?

```
from typing import Generic, TypeVar
T = TypeVar("T")
class Page(BaseModel, Generic[T]):
    items: list[T]
    total: int
    page: int
    size: int

@app.get("/users", response_model=Page[UserRead])
async def list_users(page: int = 1, size: int = 20): ...
```

Python 3.12+ also supports `class Page[T](BaseModel)`. OpenAPI then gets a concrete schema such as `Page[UserRead]`.

Q118. How do you document webhooks and callbacks in FastAPI?

Webhooks are requests your API sends to subscribers' URLs. Declare them with `@app.webhooks.post("new-order")` so they appear in OpenAPI 3.1 docs. **Callbacks** document a request your API will make back to a URL the client provided, using the `callbacks=` parameter on a route. When sending webhooks, sign payloads with HMAC, retry with backoff, and include idempotency IDs.

11. WebSockets, Streaming, Files & Forms

Q119. How do you implement a WebSocket endpoint?

```
class Manager:
    def __init__(self): self.active: list[WebSocket] = []
    async def connect(self, ws): await ws.accept(); self.active.append(ws)
    def disconnect(self, ws): self.active.remove(ws)
    async def broadcast(self, msg):
        for ws in self.active: await ws.send_text(msg)
manager = Manager()

@app.websocket("/ws/{room}")
async def ws_endpoint(ws: WebSocket, room: str):
    await manager.connect(ws)
    try:
        while True:
            data = await ws.receive_text()
            await manager.broadcast(f"{room}: {data}")
    except WebSocketDisconnect:
        manager.disconnect(ws)
```

In-memory managers only work in one process; for multiple workers/instances, use Redis pub/sub or a message broker. Authenticate via query token or cookie (browsers cannot set custom headers on WebSockets).

Q120. WebSockets vs Server-Sent Events vs long polling?

WebSockets: full-duplex, bidirectional (chat, games, collaborative editing). **SSE**: one-way server→client over HTTP, automatic reconnect, simpler, proxies-friendly (LLM token streaming, notifications). **Long polling**: fallback, higher overhead. SSE in FastAPI: `StreamingResponse(gen(), media_type="text/event-stream")` or `sse-starlette`.

Q121. How do you stream a response (e.g., LLM tokens or a large file)?

```
async def token_stream(prompt):
    async for tok in llm.astream(prompt):
        yield f"data: {tok}\n\n"

@app.post("/chat")
async def chat(req: ChatReq):
    return StreamingResponse(token_stream(req.prompt), media_type="text/event-stream")
```

For large files from disk use `FileResponse`. Disable proxy buffering (NGINX `X-Accel-Buffering: no`) for real-time streaming.

Q122. How do you handle file uploads? `bytes` vs `UploadFile`?

```
@app.post("/upload")
async def upload(file: UploadFile, description: Annotated[str, Form()]):
    contents = await file.read()
    return {"name": file.filename, "type": file.content_type, "size": len(contents)}
```

`bytes = File()` loads the whole file into memory (small files only). `UploadFile` uses a `SpooledTemporaryFile` (memory up to a limit then disk), exposes `filename`, `content_type`, async `read/write/seek/close` — better for large files. Use `list[UploadFile]` for multiple. Requires `python-multipart`. Validate type/size, sanitize filenames, never trust `content_type` alone, and store outside the web root (or in blob storage).

Q123. Why can't you send JSON body and form/file fields in the same request?

The request body has a single encoding: `application/json` or `multipart/form-data` / `application/x-www-form-urlencoded`. When files/forms are present, other fields must be `Form()` fields (you can send JSON as a string form field and parse it with `Model.model_validate_json`).

Q124. How do you serve static files and templates?

`app.mount("/static", StaticFiles(directory="static"), name="static")`; for HTML use `Jinja2Templates(directory="templates")` and return `templates.TemplateResponse(request, "page.html", {"x": 1})`.

12. Testing FastAPI Applications

Q125. How do you test FastAPI endpoints?

```
from fastapi.testclient import TestClient
client = TestClient(app)

def test_read_item():
    r = client.get("/items/1", headers={"X-Token": "abc"})
    assert r.status_code == 200
    assert r.json() == {"item_id": 1, "q": None}
```

`TestClient` (based on httpx) calls the ASGI app in-process without a running server. Use `with TestClient(app) as client:` to trigger lifespan events.

Q126. How do you write async tests?

```
import pytest
from httpx import AsyncClient, ASGITransport

@pytest.mark.anyio
async def test_root():
    async with AsyncClient(transport=ASGITransport(app=app), base_url="http://test") as ac:
        r = await ac.get("/")
    assert r.status_code == 200
```

Needed when the test itself must await async things (e.g., async DB setup).

Q127. How do you test with a database?

- Override the session dependency with one bound to a test DB (SQLite in-memory or a Postgres test container via `testcontainers`).
- Wrap each test in a transaction and roll back for isolation, or recreate tables per test session.
- Use pytest fixtures for client, DB, authenticated user, and factories (factory_boy).

Q128. How do you mock external services and authentication?

Use `app.dependency_overrides` for dependencies (auth, clients); `respx` to mock httpx calls; `unittest.mock.patch` / `pytest-mock` for functions; `freezegun` for time. Test WebSockets with `client.websocket_connect("/ws")`.

Q129. What types of tests should a FastAPI service have?

Unit tests (services, validators), integration tests (endpoints + DB), contract tests (OpenAPI schema / consumer-driven, e.g., Schemathesis property-based testing against the schema), load tests (Locust, k6), and security tests (auth bypass, input fuzzing). Measure coverage with `pytest-cov`.

13. Configuration, Deployment & Production

Q130. How do you manage configuration and settings?

```
from pydantic_settings import BaseSettings, SettingsConfigDict
class Settings(BaseSettings):
    database_url: str
    secret_key: str
    debug: bool = False
    model_config = SettingsConfigDict(env_file=".env", env_prefix="APP_")

@lru_cache
def get_settings() -> Settings: return Settings()
```

Values come from environment variables / `.env`, are type-validated at startup, and can be injected via `Depends(get_settings)` (overridable in tests). `lru_cache` avoids re-reading on each request.

Q131. How do you containerize a FastAPI app?

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY ./app ./app
RUN useradd -m appuser
USER appuser
EXPOSE 8000
CMD ["fastapi", "run", "app/main.py", "--port", "8000", "--proxy-headers"]
```

Copy requirements first for layer caching; use slim/multi-stage images; run as non-root; use exec-form CMD so signals reach the process for graceful shutdown.

Q132. Describe a production deployment architecture.

Client → CDN/WAF → load balancer / reverse proxy (NGINX, Traefik, Azure Front Door / App Gateway) handling TLS → multiple containers each running Uvicorn (Kubernetes, ECS, Azure Container Apps, App Service) → DB with connection pooling (PgBouncer), Redis cache, message queue + workers. Add health checks (`/health` liveness, `/ready` readiness), autoscaling (HPA on CPU/RPS), centralized logging, metrics (Prometheus via `prometheus-fastapi-instrumentator`), tracing (OpenTelemetry), and CI/CD.

Q133. How many workers should you run?

A common starting point is `(2 × CPU cores) + 1` for I/O-bound apps; for CPU/ML-heavy apps, roughly one worker per core (or per GPU). In Kubernetes, prefer 1 worker per pod and scale pods. Each worker has its own memory (models loaded per worker!), so memory often limits worker count.

Q134. What is `root_path` and why does it matter behind a proxy?

When a proxy serves the app under a prefix (e.g., `/api`) and strips it, the app must know the prefix to generate correct URLs for docs/OpenAPI. Set `FastAPI(root_path="/api")` or `uvicorn --root-path /api`. Also use `--proxy-headers` / `--forwarded-allow-ips` so `X-Forwarded-*` headers determine client IP and scheme.

Q135. How do you implement graceful shutdown and health checks?

Uvicorn handles SIGTERM: stops accepting connections, finishes in-flight requests (`--timeout-graceful-shutdown`), then runs lifespan shutdown. Liveness endpoint returns quickly without dependencies; readiness checks DB/model loaded so the orchestrator only routes traffic when ready.

Q136. How do you add logging and observability?

Structured JSON logs (structlog / python-json-logger) with request IDs; configure Uvicorn loggers; avoid logging secrets/PII. Metrics: request rate, latency histograms, error rates (RED). Tracing with OpenTelemetry (`opentelemetry-instrumentation-fastapi`) exported to Jaeger / Azure Monitor / Datadog. Error tracking with Sentry.

Q137. How do you make calls to downstream services resilient?

- **Timeouts** on every outbound call: `httpx.AsyncClient(timeout=httpx.Timeout(5.0, connect=2.0))`. Never rely on the default of waiting forever.
- **Retries** with exponential backoff and jitter, only for idempotent operations and transient errors (5xx, timeouts); use `tenacity` or httpx transports.
- **Circuit breaker**: after repeated failures, fail fast for a cool-down period so you don't overload a struggling dependency (`pybreaker`, `aiobreaker`).
- **Bulkheads**: limit concurrency per dependency with `asyncio.Semaphore`.
- **Fallbacks**: cached or default responses, and graceful degradation (503 + `Retry-After`).

Q138. How do FastAPI microservices communicate with each other?

Synchronous: REST over HTTP (a shared `httpx.AsyncClient`) or gRPC, which is simple but couples availability. **Asynchronous**: messaging (Kafka, RabbitMQ, Azure Service Bus, Redis Streams) with events and the outbox pattern for reliable publishing, which decouples services and absorbs spikes. Also needed: service discovery (Kubernetes DNS), an API gateway, distributed tracing (propagate `traceparent`), shared schemas and contract tests, and eventual consistency with sagas instead of distributed transactions.

Q139. How would you scale a FastAPI service to handle 10x traffic?

1. Measure first: load test and find the bottleneck (CPU, DB, external calls, event loop blocking).
2. Scale horizontally: keep the service stateless, add replicas behind a load balancer, and autoscale.
3. Database: connection pooling (PgBouncer), indexes, read replicas, query optimization, caching.
4. Move slow work to queues and workers; return 202.
5. Use async I/O, tune workers and the threadpool, use faster serialization.
6. Put a CDN and edge caching in front of public content; add rate limiting to protect the backend.
7. Observability and SLOs, so you can see the effect of each change.

14. Serving ML Models with FastAPI

Q140. How do you serve a PyTorch model with FastAPI?

```
@asynccontextmanager
async def lifespan(app):
    model = MyNet()
    model.load_state_dict(torch.load("model.pt", map_location="cpu", weights_only=True))
    model.eval()
    app.state.model = model
    yield

app = FastAPI(lifespan=lifespan)

class PredictIn(BaseModel):
    features: list[float] = Field(min_length=10, max_length=10)
class PredictOut(BaseModel):
    label: int
    prob: float

@app.post("/predict", response_model=PredictOut)
def predict(inp: PredictIn, request: Request):          # sync def -> threadpool
    x = torch.tensor(inp.features).unsqueeze(0)
    with torch.inference_mode():
        probs = request.app.state.model(x).softmax(-1)[0]
    p, idx = probs.max(0)
    return PredictOut(label=idx.item(), prob=p.item())
```

Key points: load once at startup (not per request), `eval()` + `inference_mode()`, validate inputs with Pydantic, use a sync endpoint (or offload) because inference is blocking/CPU/GPU-bound, and set threads (`torch.set_num_threads`) to avoid oversubscription with multiple workers.

Q141. How do you handle high-throughput inference? What is dynamic batching?

Dynamic batching collects concurrent requests for a few milliseconds into one batch, runs a single forward pass (much better GPU utilization), then returns individual results via futures. Implement with an `asyncio.Queue` + background consumer task, or use dedicated servers (NVIDIA Triton, TorchServe, Ray Serve, BentoML, vLLM/TGI for LLMs) behind FastAPI. Other techniques: ONNX Runtime/TensorRT, quantization, half precision, model warm-up, GPU-per-worker allocation, caching repeated predictions.

Q142. How do you handle long-running inference (e.g., video processing)?

Async job pattern: POST returns `202 Accepted` + `job_id`; job pushed to a queue (Celery/Redis, Azure Service Bus); GPU workers process and store results (blob/DB); client polls `GET /jobs/{id}`, or receives a webhook / WebSocket / SSE notification.

Q143. Why does loading a model in each worker matter, and how can you reduce memory?

Each Uvicorn/Gunicorn worker is a separate process with its own copy of the model, so 4 workers = 4× memory (and GPU memory). Mitigations: fewer workers + threadpool/batching; Gunicorn `--preload` with copy-on-write (CPU only, fragile with PyTorch); move inference into a separate model server shared by API workers; quantize/compress models.

Q144. How would you version models and do A/B or canary releases?

Store models in a registry (MLflow, Azure ML) with versions; include model version in responses and logs; route traffic by header or percentage (gateway or in-app router) between model versions; shadow-deploy new models (log predictions without serving them); monitor latency, error rate, data drift and prediction drift before full rollout.

Part II
PyTorch

15. PyTorch Fundamentals & Tensors

Q145. How do you install PyTorch and verify that the GPU works?

Use the selector on pytorch.org to get the right command for your OS, package manager and CUDA version, e.g., `pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124`. The pip wheels bundle the CUDA runtime, so you only need a compatible NVIDIA driver, not a full CUDA toolkit.

```
import torch
print(torch.__version__, torch.version.cuda)
print(torch.cuda.is_available(), torch.cuda.device_count(), torch.cuda.get_device_name(0))
print(torch.backends.mps.is_available())      # Apple Silicon
```

Q146. What are the basic attributes of a tensor?

```
x = torch.randn(2, 3, 4)
x.shape / x.size()   # torch.Size([2, 3, 4])
x.ndim / x.dim()     # 3
x.numel()            # 24 elements
x.dtype              # torch.float32
x.device             # cpu
x.requires_grad      # False
x.is_contiguous(); x.stride()   # (12, 4, 1)
x.element_size()     # bytes per element (4)
```

Q147. How do PyTorch tensors differ from NumPy arrays?

Their APIs are similar, but tensors also:

- Run on GPUs and other accelerators
- Track gradients for autograd
- Support more dtypes (bf16, fp8)
- Integrate with `nn`, `torch.compile` and distributed training

Conversion: `torch.from_numpy(a)` and `t.numpy()` share memory on the CPU. A GPU tensor or one that requires grad needs `t.detach().cpu().numpy()` first. Note that NumPy defaults to float64 while PyTorch defaults to float32.

Q148. What is the difference between `*`, `@`, `torch.mm`, `torch.matmul` and `torch.bmm`?

`*` (`torch.mul`) is **element-wise** multiplication with broadcasting. `@` / `torch.matmul` is matrix multiplication: dot product for 1-D, matrix product for 2-D, and batched with broadcasting for N-D. `torch.mm` handles strictly 2-D matrices with no broadcasting. `torch.bmm` handles strictly 3-D batches `(B, n, m) @ (B, m, p)`. Use `x.T` for 2-D transpose and `x.mT` for transposing the last two dims of a batch.

Q149. How do you get Python values out of a tensor?

`t.item()` for a single-element tensor, `t.tolist()` for nested lists, `t.numpy()` for NumPy. Each call on a GPU tensor copies data to the host and synchronizes, so avoid them in hot loops.

Q150. Explain tensor storage, strides and views.

A tensor is a **view** over a 1-D contiguous **storage** buffer, defined by:

- An offset
- A shape
- **Strides**: how many elements to skip in storage to move one step along each dim

A contiguous `(3, 4)` tensor has strides `(4, 1)`; its transpose has shape `(4, 3)` and strides `(1, 4)`, using the same storage with no copy. Slicing, `view`, `transpose`, `expand` and indexing with slices create views, and modifying a view modifies the original. Advanced indexing (index tensors, boolean masks) creates copies. Check `a.untyped_storage().data_ptr() == b.untyped_storage().data_ptr()` to see whether two tensors share memory.

Q151. What is PyTorch and what are its main components?

PyTorch is an open-source deep learning framework (Meta / Linux Foundation) offering GPU-accelerated tensors and automatic differentiation with a Pythonic, eager-execution (define-by-run) style. Components: `torch` (tensors, ops), `torch.autograd`, `torch.nn` (layers, losses), `torch.optim`, `torch.utils.data` (Dataset/DataLoader), `torch.cuda` / `mps`, `torch.distributed`, `torch.compile`, `torch.export` / `torch.onnx`, `torch.ao.quantization`, plus domain libraries (torchvision, torchaudio, torchtext legacy).

Q152. PyTorch vs TensorFlow/Keras vs JAX?

Aspect	PyTorch	TensorFlow 2 / Keras	JAX
Execution	Eager by default; `torch.compile` for graphs	Eager + `tf.function` graphs	Functional, `jit`-compiled (XLA)
Style	Pythonic, OOP (`nn.Module`)	High-level Keras API	Pure functions, explicit state
Ecosystem	Dominant in research & LLMs (HF)	Strong production (TF Serving, TFLite)	Google research, TPUs
Deployment	TorchScript/export, ONNX, ExecuTorch	SavedModel, TFLite, TF.js	Via export to TF/ONNX

Q153. What is a tensor? How do you create tensors?

A tensor is a multi-dimensional array with a `dtype`, `device`, `shape`, strides and optional gradient tracking.

```
torch.tensor([[1, 2], [3, 4]], dtype=torch.float32)    # copies data
torch.zeros(2, 3); torch.ones(2, 3); torch.empty(2, 3)  # empty = uninitialized
torch.rand(2, 3)      # U[0,1)      torch.randn(2, 3)  # N(0,1)
torch.arange(0, 10, 2); torch.linspace(0, 1, 5); torch.eye(3)
torch.full((2, 2), 7.0); torch.zeros_like(x); torch.randint(0, 10, (3,))
torch.from_numpy(arr)  # shares memory with NumPy array
torch.as_tensor(arr)   # avoids copy if possible
```

Q154. `torch.tensor` vs `torch.Tensor` vs `torch.from_numpy` vs `torch.as_tensor`?

`torch.tensor(data)` (factory function) always copies and infers dtype. `torch.Tensor(...)` is the class constructor; defaults to float32 and behaves inconsistently (`torch.Tensor(3)` creates an uninitialized length-3 tensor) — avoid. `torch.from_numpy` shares memory with the NumPy array (changes reflect in both; CPU only). `torch.as_tensor` shares memory when possible, otherwise copies.

Q155. Explain `view`, `reshape`, `permute`, `transpose`, `contiguous`.

- `view(shape)`: returns a new view sharing storage; requires compatible (usually contiguous) memory layout, else error.
- `reshape(shape)`: returns a view if possible, otherwise copies — always works.
- `transpose(d0, d1)`: swaps two dims; `permute(*dims)`: arbitrary reordering. Both return non-contiguous views (only strides change).
- `contiguous()`: returns a tensor with row-major memory layout (copies if needed); required before `view` after permute.
- Also: `squeeze` / `unsqueeze` (remove/add size-1 dims), `flatten`, `unflatten`, `expand` (broadcast view, no copy) vs `repeat` (copies).

Q156. What is broadcasting?

Rules for operating on tensors of different shapes without copying: align shapes from the trailing dimension; dimensions are compatible if equal or one of them is 1 (or missing); size-1 dims are virtually expanded. E.g., `(32, 1, 5) + (4, 5)` → `(32, 4, 5)`. Pitfall: `(N,) - (N,1)` silently produces `(N,N)`, a common bug in loss computations.

Q157. What are in-place operations? Why should you be careful with them?

Ops with trailing underscore (`x.add_(1)`, `relu_`, `x[0] = 5`) modify data in place, saving memory. Risks: they can overwrite values autograd saved for backward → `RuntimeError: one of the variables needed for gradient computation has been modified by an inplace operation`; in-place on a leaf tensor that requires grad is disallowed; and they can create subtle aliasing bugs with views.

Q158. What dtypes does PyTorch support and when do you use each?

`float32` (default training), `float16` / `bfloat16` (mixed precision; bf16 has float32's range, less precision, no loss scaling needed), `float64` (scientific precision), `int64` / `long` (class indices for `CrossEntropyLoss`, embedding indices), `int8` / `uint8` (quantization, images), `bool` (masks), `float8` (H100 training/inference), `complex64`. Convert with `x.float()`, `x.to(torch.bfloat16)`.

Q159. How do you move tensors between devices? What is device-agnostic code?

```
device = torch.device("cuda" if torch.cuda.is_available()
                      else "mps" if torch.backends.mps.is_available() else "cpu")
model.to(device); x = x.to(device, non_blocking=True)
```

All tensors in an op must be on the same device. `model.to()` moves parameters in place; `tensor.to()` returns a new tensor (must reassign). `.cpu().numpy()` to convert back (requires `.detach()` if it requires grad).

Q160. Explain indexing, slicing, masking and gather/scatter.

```
x[:, 0]; x[..., -1]; x[x > 0]            # boolean mask -> 1-D copy
torch.where(cond, a, b)                    # elementwise select
x.masked_fill(mask, float("-inf"))         # e.g., attention masks
torch.gather(logits, 1, labels.unsqueeze(1))   # pick per-row values
out.scatter_(1, idx, 1.0)                  # one-hot / write by index
torch.index_select(x, 0, idx); x[idx]      # fancy indexing (copy)
```

Q161. What are common reduction and linear algebra ops?

`sum`, `mean`, `max` (returns values+indices with `dim`), `argmax`, `topk`, `std`, `cumsum`, `norm`, `logsumexp`; use `dim` and `keepdim=True`. Linear algebra: `@` / `torch.matmul` (broadcast batched), `bmm`, `mm`, `einsum` (e.g., `"bhqd,bhkd->bhqk"`), `torch.linalg.inv/svd/qr/eigh/solve`, `cat` (existing dim) vs `stack` (new dim), `split`, `chunk`.

Q162. How do you ensure reproducibility in PyTorch?

```
torch.manual_seed(42); np.random.seed(42); random.seed(42)
torch.cuda.manual_seed_all(42)
torch.backends.cudnn.benchmark = False
torch.use_deterministic_algorithms(True)   # may need CUBLAS_WORKSPACE_CONFIG=:4096:8
```

Also seed DataLoader workers (`worker_init_fn`, `generator`). Full determinism may be slower and is not guaranteed across hardware/versions.

16. Autograd & Computational Graphs

Q163. How does autograd work?

Autograd implements reverse-mode automatic differentiation. When operations run on tensors with `requires_grad=True`, PyTorch records a dynamic computational graph (DAG) where each result has a `grad_fn` (e.g., `MulBackward0`) pointing to its inputs. Calling `loss.backward()` traverses the graph from the output to the leaves, applying the chain rule (vector-Jacobian products), and accumulates gradients into `.grad` of leaf tensors. The graph is freed after backward by default.

```
x = torch.tensor(2.0, requires_grad=True)
y = x**3 + 2*x
y.backward()
print(x.grad)   # 3*x^2 + 2 = 14
```

Q164. What is a dynamic (define-by-run) graph and why is it useful?

The graph is built on the fly during each forward pass, so it can differ per iteration. You can use native Python control flow (if/loops/recursion), variable-length inputs, and debug with print/pdb. Static graphs (TF1) are defined then executed, enabling global optimizations; PyTorch recovers these via `torch.compile`.

Q165. What are leaf tensors? Why is `.grad` None for intermediate tensors?

Leaf tensors are created by the user (not the result of a tracked op), e.g., model parameters. Only leaves with `requires_grad=True` retain `.grad` by default to save memory. For a non-leaf, call `t.retain_grad()` before backward, or use hooks.

Q166. Why do gradients accumulate and why call `optimizer.zero_grad()`?

`backward()` **adds** to existing `.grad` values. This enables gradient accumulation across micro-batches and summing gradients from multiple losses, but means you must reset gradients each step. `zero_grad(set_to_none=True)` (default in PyTorch 2) sets them to None, saving memory and a kernel.

Q167. `torch.no_grad()` vs `torch.inference_mode()` vs `model.eval()` vs `detach()`?

- `model.eval()`: switches module behavior (Dropout off, BatchNorm uses running stats). Does **not** disable gradient tracking.
- `torch.no_grad()`: disables graph recording inside the block — saves memory/compute; tensors can still later be used in autograd.
- `torch.inference_mode()`: stricter, faster version of no_grad (also skips version counters/view tracking); resulting tensors can't be used in autograd later. Best for pure inference.
- `tensor.detach()`: returns a tensor sharing storage but cut off from the graph (e.g., for target networks, logging, stopping gradient flow).

For evaluation use both `model.eval()` and `no_grad` / `inference_mode`.

Q168. What does `backward()` need for a non-scalar output?

A `gradient` argument (the vector in the vector-Jacobian product) of the same shape: `y.backward(torch.ones_like(y))`. Usually you reduce to a scalar loss first.

Q169. What is `retain_graph=True` and `create_graph=True`?

`retain_graph=True` keeps the graph's saved tensors so you can call backward again through it (e.g., multiple losses sharing parts; GAN tricks) — otherwise a second backward errors. `create_graph=True` builds a graph of the backward pass itself, enabling higher-order derivatives (gradient penalty in WGAN-GP, MAML, Hessian-vector products).

Q170. How do you compute gradients without populating `.grad`?

`torch.autograd.grad(outputs, inputs, create_graph=False)` returns gradients as a tuple, useful for gradient penalties and meta-learning. `torch.func` (functorch) offers `grad`, `vmap`, `jacrev`, `jacfwd`, `hessian` for functional transforms, per-sample gradients, and forward-mode AD.

Q171. How do you freeze parameters?

```
for p in model.backbone.parameters():
    p.requires_grad_(False)
optimizer = torch.optim.AdamW(filter(lambda p: p.requires_grad, model.parameters()), lr=1e-3)
```

Frozen layers still need `eval()` if they contain BatchNorm and you want their running stats fixed.

Q172. What is gradient checkpointing?

A memory/compute trade-off: instead of storing all intermediate activations, store only some, and recompute the rest during backward. `torch.utils.checkpoint.checkpoint(block, x, use_reentrant=False)`. Reduces activation memory (roughly from O(n) to O(√n) layers) at ~20–30% extra compute. Widely used for large transformers.

17. nn.Module & Building Models

Q173. What is `nn.Module`? Write a simple model.

Base class for all layers and models. It registers parameters, buffers and submodules assigned as attributes; provides `parameters()`, `state_dict()`, `to()`, `train()/eval()`, hooks, etc.

```
class MLP(nn.Module):
    def __init__(self, in_dim, hidden, out_dim, p=0.2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden), nn.ReLU(), nn.Dropout(p),
            nn.Linear(hidden, out_dim))
    def forward(self, x):
        return self.net(x)

model = MLP(784, 256, 10)
logits = model(x)       # call the module, not model.forward(x)
```

Q174. Why call `model(x)` instead of `model.forward(x)`?

`__call__` runs forward pre-hooks, `forward`, forward hooks and backward hook registration. Calling `forward` directly bypasses hooks (used by profilers, quantization, FSDP, etc.).

Q175. Why must you call `super().__init__()`, and why do submodules in a Python list not get registered?

`nn.Module.__init__` sets up internal dicts (`_parameters`, `_modules`, `_buffers`); without it, assigning a layer raises an error. Modules stored in a plain `list`/`dict` are invisible to `parameters()`, `.to(device)` and `state_dict` → use `nn.ModuleList` / `nn.ModuleDict` (and `nn.ParameterList`).

Q176. `nn.Sequential` vs `nn.ModuleList` vs `nn.ModuleDict`?

`Sequential`: ordered container with its own `forward` that chains modules. `ModuleList`: registered list; no forward — you iterate manually (e.g., transformer layers with skip connections). `ModuleDict`: registered dict keyed by name (multi-head models, task-specific heads).

Q177. Parameters vs buffers?

`nn.Parameter`: learnable tensor (requires grad), returned by `parameters()`, updated by optimizer. Buffers (`self.register_buffer("running_mean", t, persistent=True)`): non-learnable state saved in `state_dict` and moved with `.to()` — e.g., BatchNorm running stats, positional encodings, masks. `persistent=False` excludes it from state_dict.

Q178. What does `model.train()` vs `model.eval()` change?

Sets the `training` flag recursively. Affects layers like **Dropout** (active in train, identity in eval) and **BatchNorm** (batch statistics + update running stats in train; running stats in eval). Forgetting `eval()` at inference gives noisy, batch-dependent predictions.

Q179. What is a `state_dict`? How do you save and load models?

An `OrderedDict` mapping parameter/buffer names to tensors. Recommended:

```
torch.save(model.state_dict(), "model.pt")
model = MLP(784, 256, 10)
model.load_state_dict(torch.load("model.pt", map_location="cpu", weights_only=True))
model.eval()
```

Saving the whole model (`torch.save(model)`) pickles the class path — brittle when code moves. `weights_only=True` (default in 2.6+) prevents arbitrary code execution from untrusted pickles. Safetensors format is a safe, fast alternative. `load_state_dict(strict=False)` tolerates missing/unexpected keys (returns them).

Q180. How are weights initialized? How do you customize initialization?

Layers have defaults (e.g., `nn.Linear` uses Kaiming-uniform with a=√5). Custom:

```
def init_weights(m):
    if isinstance(m, nn.Linear):
        nn.init.kaiming_normal_(m.weight, nonlinearity="relu")
        nn.init.zeros_(m.bias)
model.apply(init_weights)
```

Xavier/Glorot for tanh/sigmoid, Kaiming/He for ReLU, orthogonal for RNNs, small normal (std 0.02) for transformers.

Q181. How do you count parameters and inspect a model?

```
total = sum(p.numel() for p in model.parameters())
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
for name, module in model.named_modules(): ...
print(model)   # or torchinfo.summary(model, input_size=(1, 3, 224, 224))
```

Q182. What is the difference between `nn.ReLU` and `F.relu`?

`torch.nn.functional` provides stateless functions; `nn` classes wrap them as modules. For parameterless ops either works; modules show up in `print(model)`, can be put in `Sequential`, and are visible to hooks/quantization. Layers with parameters (Linear, Conv) should be modules. Note `F.dropout` needs `training=self.training`.

Q183. What are common layers in `torch.nn`?

Linear, Bilinear; Conv1d/2d/3d, ConvTranspose2d; MaxPool/AvgPool/AdaptiveAvgPool; BatchNorm, LayerNorm, GroupNorm, InstanceNorm, RMSNorm; Dropout; Embedding, EmbeddingBag; RNN, LSTM, GRU; MultiheadAttention, TransformerEncoderLayer/DecoderLayer, Transformer; activations (ReLU, GELU, SiLU, LeakyReLU, Softmax); Flatten, Unflatten, Identity, PixelShuffle, Upsample.

Q184. Which activation functions does PyTorch provide and when would you use each?

Activation	Formula / property	Typical use
`nn.ReLU`	max(0, x); cheap; risk of "dying ReLU"	Default for CNNs / MLPs
`nn.LeakyReLU`, `nn.PReLU`	Small slope for x < 0 (learnable in PReLU)	Avoid dead units, GANs
`nn.GELU`	x·Φ(x), smooth	Transformers (BERT, GPT, ViT)
`nn.SiLU` (Swish)	x·σ(x)	EfficientNet, LLaMA (SwiGLU)
`nn.Sigmoid`	(0, 1); saturates → vanishing gradients	Binary / multi-label outputs, gates
`nn.Tanh`	(-1, 1), zero-centered	RNN states, GAN generator output
`nn.Softmax(dim)`	Normalizes to a probability distribution	Inference outputs, attention (not before CE loss)

Q185. Compare BatchNorm, LayerNorm, GroupNorm, InstanceNorm and RMSNorm.

- **BatchNorm**: normalizes each channel across the batch (and spatial dims). Needs reasonably large batches, and behaves differently in train and eval (running stats). Standard in CNNs.
- **LayerNorm**: normalizes across the features of each sample. Independent of batch size and identical in train and eval. Standard in transformers and RNNs.
- **GroupNorm**: normalizes groups of channels per sample. Good for small batches, e.g., detection and segmentation.
- **InstanceNorm**: normalizes per sample per channel. Used for style transfer.
- **RMSNorm**: LayerNorm without mean-centering, scaling by root-mean-square only. Cheaper; used in LLaMA and other modern LLMs.

Q186. Explain the parameters of `nn.Conv2d`.

`nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True, padding_mode="zeros")`

- Weight shape: `(out, in/groups, kH, kW)`. Input shape: `(N, C, H, W)`.
- `padding="same"` keeps the spatial size when `stride=1`.
- `stride=2` downsamples.
- `dilation` enlarges the receptive field without adding parameters.
- `groups=in_channels` gives a **depthwise** convolution; follow it with a 1×1 (pointwise) conv for a depthwise-separable convolution (MobileNet).
- A 1×1 conv mixes channels and changes channel count cheaply.

Parameter count: `out × (in/groups) × kH × kW (+ out for bias)`.

Q187. How do you replace or modify layers in an existing model (model surgery)?

```
model.fc = nn.Linear(model.fc.in_features, 5)                 # replace head
model.conv1 = nn.Conv2d(1, 64, 7, 2, 3, bias=False)          # grayscale input
for name, m in model.named_children():                        # swap all ReLU -> SiLU
    if isinstance(m, nn.ReLU): setattr(model, name, nn.SiLU())
backbone = nn.Sequential(*list(model.children())[:-1])     # drop the classifier
```

To swap modules deep in the model, recurse with `named_modules()` and `setattr` on the parent module. For multi-layer feature extraction, use `torchvision.models.feature_extraction.create_feature_extractor`. Create the optimizer **after** the surgery, so it holds the new parameters.

Q188. What are weight tying and parametrizations (weight norm, spectral norm)?

Weight tying shares one parameter between layers, e.g., a language model's input embedding and its output projection: `self.lm_head.weight = self.embed.weight`. This saves memory and often improves quality. **Parametrizations** (`torch.nn.utils.parametrize`) compute a weight from underlying parameters on every access: `parametrizations.weight_norm`, `spectral_norm` (constrains the Lipschitz constant, used in GAN discriminators) and `orthogonal`. You can register custom ones, e.g., to force symmetric matrices.

18. Loss Functions, Optimizers & LR Schedulers

Q189. What inputs does `nn.CrossEntropyLoss` expect?

Raw, unnormalized **logits** of shape `(N, C)` (or `(N, C, d1, ...)` for segmentation) and targets as class indices `(N,)` of dtype `long` (or class probabilities of shape `(N, C)`). It internally combines `LogSoftmax` + `NLLLoss` for numerical stability — do **not** apply softmax before it. Options: `weight` (class imbalance), `ignore_index` (padding, default -100), `label_smoothing`, `reduction`.

Q190. `BCELoss` vs `BCEWithLogitsLoss`?

`BCELoss` expects probabilities (after sigmoid); numerically unstable. `BCEWithLogitsLoss` takes logits and fuses sigmoid + BCE using the log-sum-exp trick — preferred. Supports `pos_weight` for imbalance. Used for binary and multi-label classification (targets float, same shape as logits).

Q191. List common loss functions and their use cases.

Loss	Use
`MSELoss` (L2)	Regression; sensitive to outliers
`L1Loss`, `SmoothL1Loss`, `HuberLoss`	Robust regression, bounding boxes
`CrossEntropyLoss`, `NLLLoss`	Multi-class classification, LM
`BCEWithLogitsLoss`	Binary / multi-label
`KLDivLoss`	Distillation, VAEs (input log-probs)
`CTCLoss`	Speech/OCR unaligned sequences
`MarginRankingLoss`, `TripletMarginLoss`, `CosineEmbeddingLoss`	Metric learning, ranking
Focal / Dice (custom or torchvision.ops)	Imbalanced detection / segmentation

Q192. How do you write a custom loss?

```
class FocalLoss(nn.Module):
    def __init__(self, gamma=2.0, alpha=0.25):
        super().__init__(); self.gamma, self.alpha = gamma, alpha
    def forward(self, logits, targets):
        bce = F.binary_cross_entropy_with_logits(logits, targets, reduction="none")
        p_t = torch.exp(-bce)
        return (self.alpha * (1 - p_t) ** self.gamma * bce).mean()
```

Any differentiable tensor ops work; autograd handles gradients.

Q193. Compare SGD, Momentum, Adam and AdamW.

- **SGD**: `w -= lr * g`. Simple; with momentum (0.9) and Nesterov often generalizes best for CNNs.
- **Momentum**: accumulates velocity to dampen oscillations and speed through ravines.
- **RMSprop / Adagrad**: per-parameter adaptive learning rates.
- **Adam**: momentum (1st moment) + RMSprop (2nd moment) with bias correction; fast convergence, less tuning.
- **AdamW**: Adam with *decoupled* weight decay (applied directly to weights instead of via gradient), which regularizes correctly; standard for transformers.

Others: LAMB/LARS (large batch), Adafactor (memory-efficient), Lion, Sophia; 8-bit Adam (bitsandbytes).

Q194. How do you use parameter groups (different LR / weight decay per layer)?

```
decay, no_decay = [], []
for n, p in model.named_parameters():
    (no_decay if p.ndim < 2 or "norm" in n else decay).append(p)
opt = torch.optim.AdamW([
    {"params": decay, "weight_decay": 0.01},
    {"params": no_decay, "weight_decay": 0.0}], lr=3e-4)
# fine-tuning: lower LR for backbone, higher for new head
opt = torch.optim.SGD([{"params": model.backbone.parameters(), "lr": 1e-4},
                       {"params": model.head.parameters()}], lr=1e-2, momentum=0.9)
```

Q195. What learning rate schedulers exist and how are they used?

`StepLR`, `MultiStepLR`, `ExponentialLR`, `CosineAnnealingLR`, `CosineAnnealingWarmRestarts`, `OneCycleLR`, `LinearLR` (warmup), `SequentialLR` (chain warmup + cosine), `ReduceLROnPlateau` (step with a metric), `LambdaLR` (custom). Call `scheduler.step()` **after** `optimizer.step()` — per epoch or per iteration depending on the scheduler (OneCycle/warmup are per iteration). Warmup stabilizes early training for transformers/large batches.

Q196. What is gradient clipping and when do you use it?

`torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)` rescales gradients if their global norm exceeds a threshold; `clip_grad_value_` clips each element. Prevents exploding gradients in RNNs/transformers. With AMP, call `scaler.unscale_(optimizer)` before clipping.

Q197. How do you implement L1/L2 regularization?

L2: `weight_decay` in the optimizer (prefer AdamW for adaptive optimizers). L1: add manually: `loss = loss + lam * sum(p.abs().sum() for p in model.parameters())`. Other regularizers: dropout, data augmentation, label smoothing, early stopping, stochastic depth, mixup/cutmix.

Q198. How do you choose a good learning rate?

Start from known defaults: AdamW 1e-3 to 3e-4 for training from scratch, 1e-5 to 5e-5 for fine-tuning transformers; SGD 0.1 with momentum for ResNets at batch 256. Run an **LR range test**: increase the LR exponentially over a few hundred steps, plot loss against LR, and pick a value somewhat below where the loss falls fastest, before it diverges. Combine it with warmup and a decay schedule (cosine or one-cycle). Tune the LR first; it is the most important hyperparameter.

Q199. What are EMA of weights and Stochastic Weight Averaging (SWA)?

Both average model weights to reach flatter minima that generalize better. **EMA** keeps a shadow copy updated each step (`ema = decay·ema + (1-decay)·w`, e.g., decay 0.999) and is used for evaluation; it is common in diffusion models, YOLO and semi-supervised learning. **SWA** averages weights from several points late in training, typically with a constant or cyclic LR.

```
from torch.optim.swa_utils import AveragedModel, get_ema_multi_avg_fn, update_bn
ema_model = AveragedModel(model, multi_avg_fn=get_ema_multi_avg_fn(0.999))
ema_model.update_parameters(model)     # after each optimizer.step()
update_bn(train_loader, ema_model)     # recompute BatchNorm stats for SWA
```

Q200. How do you combine multiple losses (multi-task learning)?

Use a weighted sum, `loss = w1*l1 + w2*l2`, and call a single `backward()`. Tune the weights so no task dominates; check each task's gradient magnitude. Adaptive alternatives: uncertainty weighting (learnable log-variances per task), GradNorm, and PCGrad for conflicting gradients. Log each loss separately.

19. Datasets, DataLoaders & Transforms

Q201. Explain `Dataset` and `DataLoader`. Write a custom Dataset.

```
class ImageCsvDataset(Dataset):
    def __init__(self, df, root, transform=None):
        self.df, self.root, self.transform = df.reset_index(drop=True), root, transform
    def __len__(self):
        return len(self.df)
    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        img = Image.open(os.path.join(self.root, row.path)).convert("RGB")
        if self.transform: img = self.transform(img)
        return img, torch.tensor(row.label, dtype=torch.long)

loader = DataLoader(ds, batch_size=64, shuffle=True, num_workers=4,
                    pin_memory=True, drop_last=True, persistent_workers=True)
```

Map-style Dataset implements `__len__` + `__getitem__`. **Iterable-style** (`IterableDataset`) implements `__iter__` for streams/huge data. DataLoader handles batching, shuffling, sampling, multiprocess loading, collation and pinning. Load lazily in `__getitem__` rather than loading everything in `__init__`.

Q202. What do `num_workers`, `pin_memory`, `prefetch_factor` and `persistent_workers` do?

- `num_workers`: number of subprocesses loading data in parallel (0 = main process). Start around number of CPU cores / GPUs; tune empirically.
- `pin_memory=True`: places batches in page-locked memory → faster host-to-GPU copies and enables `non_blocking=True`.
- `prefetch_factor`: batches pre-loaded per worker (default 2).
- `persistent_workers=True`: keep workers alive between epochs (avoids startup cost).

On Windows/macOS (spawn), guard the entry point with `if __name__ == "__main__":` and ensure the dataset is picklable.

Q203. What is `collate_fn` and when do you need a custom one?

It merges a list of samples into a batch (default stacks tensors). Custom is needed for variable-length sequences (padding), object detection targets (lists of boxes), or skipping corrupted samples.

```
def collate(batch):
    seqs, labels = zip(*batch)
    lengths = torch.tensor([len(s) for s in seqs])
    padded = nn.utils.rnn.pad_sequence(seqs, batch_first=True, padding_value=0)
    return padded, lengths, torch.tensor(labels)
```

Q204. What samplers are available? How do you handle class imbalance?

`SequentialSampler`, `RandomSampler`, `SubsetRandomSampler`, `WeightedRandomSampler`, `BatchSampler`, `DistributedSampler`. Imbalance: `WeightedRandomSampler(weights=1/class_freq[labels], num_samples=len(ds), replacement=True)` (oversampling), class weights in the loss, focal loss, or augmentation of minority classes. `sampler` and `shuffle=True` are mutually exclusive.

Q205. How do you use torchvision transforms and data augmentation?

```
from torchvision.transforms import v2
train_tf = v2.Compose([
    v2.RandomResizedCrop(224), v2.RandomHorizontalFlip(),
    v2.ColorJitter(0.2, 0.2, 0.2), v2.ToImage(), v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])])
val_tf = v2.Compose([v2.Resize(256), v2.CenterCrop(224), v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True), v2.Normalize(...)])
```

Augment only training data. `transforms.v2` jointly transforms images, boxes, masks and keypoints. Also: RandAugment, AutoAugment, MixUp/CutMix (`v2.MixUp`), Albumentations, GPU augmentation (Kornia).

Q206. How do you split a dataset into train/val?

`torch.utils.data.random_split(ds, [0.8, 0.2], generator=torch.Generator().manual_seed(42))` or `Subset(ds, indices)` with stratified indices from scikit-learn. Use separate dataset instances if train/val need different transforms.

Q207. How do you handle datasets that don't fit in memory?

Lazy loading in `__getitem__`; memory-mapped arrays (`np.memmap`), HDF5/LMDB/Zarr; `IterableDataset` streaming from disk/cloud (WebDataset tar shards, Hugging Face `datasets` streaming, Mosaic StreamingDataset); shard across workers via `get_worker_info()`.

20. Training Loop, Evaluation & Checkpointing

Q208. Define epoch, batch, batch size, iteration and step.

A **batch** is a subset of samples processed together in one forward/backward pass. **Batch size** is the number of samples in that batch. An **iteration** or **step** is one parameter update; with gradient accumulation, one step covers several micro-batches. An **epoch** is one full pass over the training set. Iterations per epoch = `ceil(N / batch_size)`, i.e., `len(loader)`. Example: 50,000 images with batch size 100 gives 500 iterations per epoch.

Q209. What are the five essential steps of training in PyTorch?

1. **Forward pass**: `pred = model(x)`.
2. **Compute the loss**: `loss = criterion(pred, y)`.
3. **Zero the gradients**: `optimizer.zero_grad()`.
4. **Backward pass**: `loss.backward()` computes gradients.
5. **Update the parameters**: `optimizer.step()`.

Everything else is built around these steps: device moves, AMP, clipping, scheduling, logging.

Q210. Write a standard PyTorch training and validation loop.

```
def train_one_epoch(model, loader, criterion, optimizer, scheduler, device):
    model.train()
    total, correct, running = 0, 0, 0.0
    for x, y in loader:
        x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
        optimizer.zero_grad(set_to_none=True)   # 1. reset grads
        logits = model(x)                       # 2. forward
        loss = criterion(logits, y)             # 3. loss
        loss.backward()                         # 4. backward
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()                        # 5. update
        scheduler.step()                        # per-iteration scheduler
        running += loss.item() * x.size(0)
        correct += (logits.argmax(1) == y).sum().item(); total += x.size(0)
    return running / total, correct / total

@torch.no_grad()
def evaluate(model, loader, criterion, device):
    model.eval()
    total, correct, running = 0, 0, 0.0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        logits = model(x)
        running += criterion(logits, y).item() * x.size(0)
        correct += (logits.argmax(1) == y).sum().item(); total += x.size(0)
    return running / total, correct / total
```

Q211. Why use `loss.item()` when logging?

`.item()` converts a 1-element tensor to a Python number, detaching it. Accumulating `total_loss += loss` keeps the entire graph of every iteration alive → memory leak. Note `.item()` forces a GPU sync; for max speed accumulate detached tensors and sync occasionally.

Q212. How do you implement gradient accumulation?

```
accum = 4
for i, (x, y) in enumerate(loader):
    loss = criterion(model(x), y) / accum     # scale so gradients average
    loss.backward()
    if (i + 1) % accum == 0:
        optimizer.step(); optimizer.zero_grad()
```

Simulates a batch size of `accum × batch_size` when GPU memory is limited. BatchNorm still sees the small micro-batch. In DDP, use `model.no_sync()` on non-update steps to skip redundant all-reduce.

Q213. How do you save and resume a full training checkpoint?

```
torch.save({"epoch": epoch, "model": model.state_dict(),
            "optimizer": optimizer.state_dict(), "scheduler": scheduler.state_dict(),
            "scaler": scaler.state_dict(), "best_metric": best,
            "rng": torch.get_rng_state()}, "ckpt.pt")

ckpt = torch.load("ckpt.pt", map_location=device)
model.load_state_dict(ckpt["model"]); optimizer.load_state_dict(ckpt["optimizer"])
scheduler.load_state_dict(ckpt["scheduler"]); start_epoch = ckpt["epoch"] + 1
```

Optimizer state (Adam moments) is needed for proper resumption. For DDP save from rank 0 only (`model.module.state_dict()`); for FSDP use `torch.distributed.checkpoint`.

Q214. How do you implement early stopping?

Track the best validation metric; save a checkpoint when it improves; increment a counter otherwise; stop when the counter exceeds `patience`. Restore the best weights at the end. Libraries like Lightning provide `EarlyStopping` callbacks.

Q215. How do you track experiments and metrics?

TensorBoard (`torch.utils.tensorboard.SummaryWriter`), Weights & Biases, MLflow, Neptune; metrics via `torchmetrics` (handles distributed accumulation correctly). Log losses, LR, gradient norms, metrics, sample predictions, hyperparameters, git commit and seeds.

Q216. What is the order of `optimizer.step()`, `scheduler.step()` and `zero_grad()`?

`zero_grad()` → forward → loss → `backward()` → (clip) → `optimizer.step()` → `scheduler.step()`. `zero_grad` can also come right after `step`; it just must happen before the next backward. Calling the scheduler before the optimizer skips the first LR value (PyTorch warns).

Q217. How do you compute evaluation metrics correctly?

Accumulate predictions or counts over the **whole** validation set, then compute the metric. Averaging per-batch F1 or AUC is wrong, because those metrics aren't decomposable and the last batch may be smaller. `torchmetrics` handles this, including syncing across GPUs:

```
f1 = torchmetrics.F1Score(task="multiclass", num_classes=10, average="macro").to(device)
for x, y in val_loader: f1.update(model(x).argmax(1), y)
print(f1.compute()); f1.reset()
```

Q218. How do you tune hyperparameters for a PyTorch model?

Tune the learning rate first, then batch size, weight decay, dropout, depth/width and augmentation strength. Use random search or Bayesian optimization (Optuna, Ray Tune) rather than grid search, with early pruning of bad trials (median or Hyperband pruners). Search the LR on a log scale and evaluate on the validation set only. Fix seeds and log everything; scale up only the best configurations.

```
def objective(trial):
    lr = trial.suggest_float("lr", 1e-5, 1e-2, log=True)
    wd = trial.suggest_float("wd", 1e-6, 1e-1, log=True)
    return train_and_eval(lr, wd, trial)     # call trial.report + trial.should_prune inside
optuna.create_study(direction="maximize",
pruner=optuna.pruners.MedianPruner()).optimize(objective, 50)
```

21. GPU, Mixed Precision & Performance

Q219. What is Automatic Mixed Precision (AMP) and how do you use it?

AMP runs suitable ops (matmul, conv) in float16/bfloat16 for speed (Tensor Cores) and memory savings, while keeping precision-sensitive ops (reductions, softmax, losses) in float32. With fp16, a `GradScaler` multiplies the loss to avoid gradient underflow, then unscales before the update and skips steps with inf/NaN.

```
scaler = torch.amp.GradScaler("cuda")
for x, y in loader:
    optimizer.zero_grad()
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        loss = criterion(model(x), y)
    scaler.scale(loss).backward()
    scaler.unscale_(optimizer); clip_grad_norm_(model.parameters(), 1.0)
    scaler.step(optimizer); scaler.update()
```

With `bfloat16` (Ampere+), no scaler is needed thanks to its wide exponent range.

Q220. How do you diagnose and fix CUDA out-of-memory (OOM) errors?

- Reduce batch size; use gradient accumulation.
- Mixed precision (bf16/fp16); gradient checkpointing.
- Avoid keeping graphs alive (`loss.item()` , `detach()` for logged tensors); use `no_grad` in eval.
- Delete unused tensors, `torch.cuda.empty_cache()` (releases cached blocks to other processes; doesn't fix leaks).
- Use memory-efficient optimizers (8-bit Adam, Adafactor), FSDP/ZeRO sharding, CPU offload, LoRA for fine-tuning.
- Inspect with `torch.cuda.memory_summary()` , `max_memory_allocated()` , memory snapshots (`torch.cuda.memory._record_memory_history`).
- Set `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` to reduce fragmentation.

Q221. What are common ways to speed up PyTorch training?

- Fix data loading bottlenecks: `num_workers` , `pin_memory` , faster decoding, pre-processed/cached data.
- AMP (bf16/fp16), TF32 (`torch.backends.cuda.matmul.allow_tf32 = True` or `torch.set_float32_matmul_precision("high")`).
- `torch.compile(model)` ; fused optimizers (`AdamW(fused=True)`); fused attention (`F.scaled_dot_product_attention` → FlashAttention).
- `cudnn.benchmark = True` for fixed input sizes; channels_last memory format for CNNs.
- Avoid host-device syncs in the loop (`.item()` , `.cpu()` , printing, `nonzero`).
- Larger batches with LR scaling; multi-GPU DDP.
- CUDA Graphs for small, static workloads (`mode="reduce-overhead"`).

Q222. How do you profile PyTorch code?

```
from torch.profiler import profile, ProfilerActivity, schedule
with profile(activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
             schedule=schedule(wait=1, warmup=1, active=3),
             on_trace_ready=torch.profiler.tensorboard_trace_handler("./log"),
             record_shapes=True, profile_memory=True) as prof:
    for step, batch in enumerate(loader):
        train_step(batch); prof.step()
print(prof.key_averages().table(sort_by="cuda_time_total", row_limit=10))
```

CUDA ops are asynchronous — when timing manually, call `torch.cuda.synchronize()` or use CUDA events. Also: `nvidia-smi`, Nsight Systems, GPU utilization to detect data-loading stalls.

Q223. Why are CUDA operations asynchronous and what forces synchronization?

Kernels are queued on a CUDA stream and the Python thread continues, overlapping CPU and GPU work. Synchronization happens on `.item()`, `.cpu()`, `.numpy()`, `print(tensor)`, `torch.cuda.synchronize()`, data-dependent shapes (`nonzero`, boolean masking), and non-pinned copies. Errors may be reported at a later line; set `CUDA_LAUNCH_BLOCKING=1` to debug.

Q224. What is `channels_last` memory format?

Stores NCHW tensors physically as NHWC. Convolutions on Tensor Cores (especially with AMP) run faster in this layout. Use `model.to(memory_format=torch.channels_last)` and `x.to(memory_format=torch.channels_last)`.

Q225. How do you use multiple GPUs on a single machine? Why prefer DDP over DataParallel?

`nn.DataParallel`: single process, multi-thread; replicates the model every iteration, scatters inputs, gathers outputs on GPU 0 — GIL contention and imbalanced memory; legacy. `DistributedDataParallel`: one process per GPU, each with a model replica; gradients are all-reduced in buckets overlapped with backward. Faster, scales to multiple nodes, recommended even for a single node.

Q226. What are CUDA streams and events?

A **stream** is a queue of GPU operations that execute in order. Operations on different streams can overlap, e.g., copying the next batch while computing on the current one. PyTorch uses one default stream per device; `torch.cuda.Stream()` with `with torch.cuda.stream(s):` runs work on another stream, and you sync with `wait_stream` or `record_stream`. **Events** mark points in a stream for synchronization and accurate GPU timing:

```
start, end = torch.cuda.Event(enable_timing=True), torch.cuda.Event(enable_timing=True)
start.record(); y = model(x); end.record(); torch.cuda.synchronize()
print(start.elapsed_time(end), "ms")
```

Q227. How do you estimate the GPU memory needed for training?

With mixed-precision Adam, model state needs roughly 16 bytes per parameter:

- fp32 master weights: 4 bytes
- Adam moments (m and v): 8 bytes
- Gradients and bf16 weights: about 4 bytes

A 7B model therefore needs about 112 GB before counting **activations**, which scale with batch size × sequence length × hidden size × layers, plus framework overhead. Inference needs roughly 2 bytes per parameter in bf16 (14 GB for 7B), or 0.5 bytes in 4-bit, plus the KV-cache. This explains why FSDP, checkpointing, LoRA and quantization exist.

22. Distributed & Large-Scale Training

Q228. Explain how DistributedDataParallel (DDP) works and how to set it up.

```
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

def main():
    dist.init_process_group("nccl")               # env vars set by torchrun
    local_rank = int(os.environ["LOCAL_RANK"]); torch.cuda.set_device(local_rank)
    model = Net().cuda(local_rank)
    model = DDP(model, device_ids=[local_rank])
    sampler = DistributedSampler(train_ds, shuffle=True)
    loader = DataLoader(train_ds, batch_size=64, sampler=sampler, num_workers=4)
    for epoch in range(E):
        sampler.set_epoch(epoch)                  # different shuffle each epoch
        for x, y in loader: ...                   # normal training step
    if dist.get_rank() == 0: torch.save(model.module.state_dict(), "m.pt")
    dist.destroy_process_group()

# torchrun --nproc_per_node=4 --nnodes=1 train.py
```

Each process computes forward/backward on its data shard; during backward, gradient buckets are averaged with **all-reduce** (ring/tree via NCCL), so all replicas apply identical updates. Effective batch = per-GPU batch × world size (scale LR accordingly). Use `SyncBatchNorm` if per-GPU batches are small.

Q229. Define world size, rank, local rank, backend.

World size: total number of processes. **Rank**: global process ID (0 is usually the "main" for logging/saving). **Local rank**: process ID within a node (maps to GPU index). **Backend**: communication library — NCCL (GPU, fastest), Gloo (CPU), MPI. `torchrun` sets `RANK`, `LOCAL_RANK`, `WORLD_SIZE`, `MASTER_ADDR`, `MASTER_PORT` and supports elastic/fault-tolerant training.

Q230. What are collective communication operations?

`all_reduce` (sum/avg across ranks, result on all — gradient sync), `broadcast` (from one rank to all — initial weights), `all_gather` (collect tensors from all), `reduce_scatter` (reduce then shard — used by FSDP/ZeRO), `gather`, `scatter`, `barrier` (synchronize).

Q231. What are data, tensor, pipeline and sequence parallelism?

- **Data parallelism**: replicate model, split data (DDP).
- **Fully Sharded Data Parallel (FSDP / ZeRO-3)**: shard parameters, gradients and optimizer states across GPUs; all-gather parameters just in time for each layer's forward/backward. Enables models larger than one GPU's memory.
- **Tensor parallelism**: split individual layers (matmuls) across GPUs (Megatron-LM); needs fast interconnect (NVLink).
- **Pipeline parallelism**: split layers into stages on different GPUs; micro-batches fill the pipeline to reduce bubbles (GPipe, 1F1B).
- **Sequence/context parallelism**: split long sequences across devices.
- 3D parallelism combines data + tensor + pipeline for very large LLMs.

Q232. Explain ZeRO stages and FSDP.

ZeRO (DeepSpeed) removes redundancy in data parallelism. **Stage 1**: shard optimizer states. **Stage 2**: + gradients. **Stage 3**: + parameters. Memory per GPU drops roughly linearly with GPU count at the cost of more communication. PyTorch's native equivalent is FSDP (`FullyShardedDataParallel`, and FSDP2 via `fully_shard`), with options for mixed precision, CPU offload, activation checkpointing and auto-wrap policies (wrap each transformer block).

Q233. What tools simplify distributed training?

PyTorch Lightning / Fabric, Hugging Face Accelerate & Trainer, DeepSpeed, torchtitan, Megatron-LM, Ray Train, Horovod. They abstract launching, DDP/FSDP wrapping, AMP, checkpointing and logging.

Q234. How do you scale the learning rate with batch size?

Linear scaling rule: multiply LR by `k` when batch size grows by `k` (SGD), with warmup to avoid early instability. For Adam, square-root scaling is often used. Very large batches may need LARS/LAMB.

Q235. What are DTensor and DeviceMesh?

`DeviceMesh` describes a multi-dimensional layout of devices, e.g., `init_device_mesh("cuda", (2, 4), mesh_dim_names=("dp", "tp"))` for 2-way data parallelism × 4-way tensor parallelism. `DTensor` is a tensor sharded or replicated across a mesh according to placements (`Shard(dim)`, `Replicate()`, `Partial()`), and PyTorch inserts the required collectives automatically. They underpin FSDP2 (`fully_shard`), the tensor-parallel API (`parallelize_module` with `ColwiseParallel` / `RowwiseParallel`) and 2D/3D parallelism in torchtitan.

Q236. How do you initialize a model too large for CPU or GPU memory?

Create it on the **meta device**, where tensors have shapes but no storage: `with torch.device("meta"): model = Big()`. Then shard it (FSDP) and materialize only the local shards with `model.to_empty(device="cuda")`, followed by initialization or checkpoint loading. Hugging Face's `device_map="auto"` / `low_cpu_mem_usage` use the same idea.

Q237. Distributed training hangs or crashes. How do you debug it?

- Hangs are usually caused by ranks executing **different collectives**: data-dependent branches, uneven numbers of batches (use `drop_last` or the `Join` context), or only rank 0 calling a collective, e.g., during evaluation.
- Set `NCCL_DEBUG=INFO`, `TORCH_DISTRIBUTED_DEBUG=DETAIL` and `TORCH_NCCL_ASYNC_ERROR_HANDLING=1`, and use a timeout in `init_process_group`. The flight recorder (`TORCH_NCCL_TRACE_BUFFER_SIZE`) dumps the last collectives on timeout.
- "Expected to have finished reduction" means some parameters received no gradient. Fix the model, or pass `find_unused_parameters=True` (slower).
- Check network interfaces (`NCCL_SOCKET_IFNAME`), firewall rules and `MASTER_ADDR/PORT`.

23. Advanced: Custom Autograd, Hooks, torch.compile & TorchScript

Q238. How do you write a custom autograd Function?

```
class ClampGrad(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, lo, hi):
        ctx.save_for_backward(x); ctx.lo, ctx.hi = lo, hi
        return x.clamp(lo, hi)
    @staticmethod
    def backward(ctx, grad_out):
        (x,) = ctx.saved_tensors
        mask = (x >= ctx.lo) & (x <= ctx.hi)
        return grad_out * mask, None, None    # one grad per forward input

y = ClampGrad.apply(x, -1.0, 1.0)
```

Used for non-differentiable ops (straight-through estimators), memory-efficient custom gradients, or wrapping custom CUDA/Triton kernels. Verify with `torch.autograd.gradcheck` (double precision).

Q239. What are hooks in PyTorch? Give use cases.

- **Tensor hooks**: `t.register_hook(lambda g: g * 0.5)` — inspect/modify gradients.
- **Forward hooks**: `module.register_forward_hook(fn(module, inp, out))` — extract intermediate features/activations, feature maps for Grad-CAM, debugging NaNs.
- **Forward pre-hooks**: modify inputs before forward.
- **Full backward hooks**: `register_full_backward_hook` — inspect gradients w.r.t. module inputs/outputs.
- State-dict hooks for compatibility when loading old checkpoints.

They return a handle; call `handle.remove()` to avoid leaks.

```
feats = {}
h = model.layer4.register_forward_hook(lambda m, i, o: feats.__setitem__("l4", o.detach()))
```

Q240. What is `torch.compile` and how does it work?

Introduced in PyTorch 2.0: `model = torch.compile(model)` JIT-compiles the model for speed (often 1.3–2x). Pipeline:

- **TorchDynamo** captures Python bytecode into FX graphs safely, inserting **graph breaks** for unsupported Python code and falling back to eager.
- **AOTAutograd** traces the backward graph ahead of time.
- **PrimTorch** decomposes ~2000 ops into a smaller primitive set.
- **TorchInductor** generates fused kernels (Triton on GPU, C++/OpenMP on CPU).

Modes: `"default"`, `"reduce-overhead"` (CUDA graphs), `"max-autotune"`. Use `dynamic=True` for varying shapes; `fullgraph=True` to error on graph breaks. First call is slow (compilation); recompiles on shape/dtype/guard changes. Debug with `TORCH_LOGS="graph_breaks,recompiles"`.

Q241. What is TorchScript? Tracing vs scripting?

TorchScript is an intermediate representation that serializes models for execution without Python (C++ `libtorch`, mobile). **Tracing** (`torch.jit.trace(model, example)`) records ops executed for one example input — data-dependent control flow is frozen (silent bugs). **Scripting** (`torch.jit.script`) compiles Python source, preserving control flow, but supports a subset of Python. TorchScript is now in maintenance mode; `torch.export` is the recommended path forward.

Q242. What is `torch.export`?

Produces a sound, whole-graph `ExportedProgram` (ATen-level FX graph) ahead of time, with no graph breaks allowed and explicit dynamic shape specifications (`Dim`). It is the foundation for AOTInductor (compiled .so for server deployment), ExecuTorch (mobile/edge) and the new ONNX exporter (`torch.onnx.export(..., dynamo=True)`).

Q243. What is FX?

`torch.fx` is a toolkit for capturing and transforming programs as Python-level graphs (`symbolic_trace` → `GraphModule`). Used for operator fusion, quantization (FX graph mode), feature extraction (`torchvision.models.feature_extraction`), and custom compiler passes.

Q244. How do you write custom CUDA/C++ extensions or Triton kernels?

C++/CUDA: `torch.utils.cpp_extension` (`load_inline` or setuptools `CUDAExtension`), registering ops with `TORCH_LIBRARY` so they work with `torch.compile`. Triton: write Python-like GPU kernels with `@triton.jit`; wrap them with `torch.library.triton_op` / `custom_op`. Used for fused attention, custom activations, quantized matmuls.

Q245. What is `torch.func` (functorch) and `vmap`?

JAX-like composable function transforms. `vmap` auto-vectorizes a function over a batch dimension (no manual batching); `grad` / `jacrev` / `hessian` compute derivatives functionally; `functional_call(model, params, x)` runs a module with given parameters. Use cases: per-sample gradients (differential privacy), ensembling, meta-learning, Jacobians for physics-informed nets.

Q246. How does PyTorch execute an operation internally (dispatcher, ATen)?

A Python call such as `torch.add(a, b)` goes through C++ bindings into the **dispatcher**. The dispatcher picks a kernel based on the inputs' **dispatch keys**, processed in priority order:

- Autograd: records `grad_fn`
- Autocast: casts dtypes
- Functionalization and Python modes
- The backend: CPU, CUDA, MPS, Meta, sparse, quantized, etc.

The actual kernels live in **ATen**, the C++ tensor library. This layering is how one API supports many devices, how subclasses such as DTensor work, and how `torch.library` custom ops plug in.

Q247. What are saved-tensor hooks and activation offloading?

`torch.autograd.graph.saved_tensors_hooks(pack, unpack)` intercepts tensors that autograd saves for backward. You can compress them, quantize them, or move them to CPU (`torch.autograd.graph.save_on_cpu(pin_memory=True)`) and bring them back during backward. This trades PCIe bandwidth for GPU memory and complements activation checkpointing.

Q248. What is FlexAttention?

`torch.nn.attention.flex_attention` lets you express attention variants (causal, sliding window, ALiBi, document masking, soft-capping) as small Python `score_mod` / `mask_mod` functions. `torch.compile` then generates fused FlashAttention-style kernels, and block masks skip fully masked blocks. You get custom attention without writing CUDA.

24. Model Export, Quantization, Pruning & Deployment

Q249. How do you export a PyTorch model to ONNX?

```
model.eval()
dummy = torch.randn(1, 3, 224, 224)
torch.onnx.export(model, (dummy,), "model.onnx",
                  input_names=["input"], output_names=["logits"],
                  dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},
                  opset_version=17)
# verify
sess = onnxruntime.InferenceSession("model.onnx")
out = sess.run(None, {"input": dummy.numpy()})
```

ONNX is a framework-neutral format runnable by ONNX Runtime (CPU/GPU/DirectML), TensorRT, OpenVINO. Compare outputs with PyTorch (`np.testing.assert_allclose`). Newer: `torch.onnx.export(..., dynamo=True)` built on `torch.export`.

Q250. What is quantization? Explain dynamic, static and quantization-aware training.

Representing weights/activations in lower precision (int8, int4, fp8) to shrink models (~4x for int8) and speed up inference with small accuracy loss.

- **Dynamic (post-training)**: weights quantized ahead of time, activations quantized on the fly. Easy; great for LSTMs/Linear-heavy models on CPU. `torch.ao.quantization.quantize_dynamic(model, {nn.Linear}, dtype=torch.qint8)`.
- **Static (post-training)**: weights and activations quantized; requires calibration data to compute activation ranges (observers); fuse Conv-BN-ReLU first. Best CPU speedups for CNNs.
- **Quantization-aware training (QAT)**: inserts fake-quant ops during training/fine-tuning so the model learns to tolerate quantization noise; best accuracy.

Modern stack: `torchao` (int8/int4 weight-only, fp8, PT2E quantization), bitsandbytes/GPTQ/AWQ for LLMs.

Q251. What is pruning?

Removing weights (unstructured: individual weights by magnitude — sparse but needs special hardware to speed up) or whole channels/heads/layers (structured: real speedups).
`torch.nn.utils.prune.l1_unstructured(module, "weight", amount=0.3)`, then `prune.remove` to make it permanent. Usually followed by fine-tuning; 2:4 semi-structured sparsity is accelerated on Ampere+ GPUs.

Q252. What is knowledge distillation and how do you implement it in PyTorch?

Train a small student to mimic a large teacher's softened outputs.

```
T = 4.0
with torch.no_grad(): t_logits = teacher(x)
s_logits = student(x)
kd = F.kl_div(F.log_softmax(s_logits / T, -1), F.softmax(t_logits / T, -1),
              reduction="batchmean") * T * T
loss = alpha * kd + (1 - alpha) * F.cross_entropy(s_logits, y)
```

Q253. What are the options for deploying PyTorch models?

- **Python services**: FastAPI/Flask with eager or compiled model; Ray Serve; BentoML; LitServe.
- **Model servers**: TorchServe (handlers, batching, metrics — now limited maintenance), NVIDIA Triton (multi-framework, dynamic batching, ONNX/TensorRT backends), KServe on Kubernetes, Azure ML / SageMaker endpoints.
- **LLM servers**: vLLM (PagedAttention, continuous batching), TGI, SGLang, TensorRT-LLM.
- **Compiled**: ONNX Runtime, TensorRT, OpenVINO, AOTInductor.
- **Edge/mobile**: ExecuTorch (successor to PyTorch Mobile), Core ML, TFLite via conversion.

Q254. What should you consider for inference optimization?

`eval()` + `inference_mode()`; batch requests; half precision / int8; `torch.compile` or TensorRT; fuse layers (Conv+BN); warm up the model (first calls trigger cuDNN autotune/compilation); pin threads (`torch.set_num_threads`); avoid CPU↔GPU transfers per request; KV-cache for autoregressive models; monitor p50/p95/p99 latency and throughput.

25. Common Architectures in PyTorch (CNN, RNN, Transformer)

Q255. Implement a simple CNN for image classification. How do you compute output sizes?

```
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d(2))
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(64, num_classes)
    def forward(self, x):                  # x: (N, 3, H, W)
        x = self.pool(self.features(x))    # (N, 64, 1, 1)
        return self.fc(torch.flatten(x, 1))
```

Output size: `floor((W + 2P - D·(K-1) - 1) / S) + 1`. `AdaptiveAvgPool2d` makes the head independent of input resolution. Use `bias=False` on convs followed by BatchNorm.

Q256. Implement a residual block.

```
class ResBlock(nn.Module):
    def __init__(self, c_in, c_out, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(c_in, c_out, 3, stride, 1, bias=False); self.bn1 =
nn.BatchNorm2d(c_out)
        self.conv2 = nn.Conv2d(c_out, c_out, 3, 1, 1, bias=False);     self.bn2 =
nn.BatchNorm2d(c_out)
        self.short = nn.Identity() if stride == 1 and c_in == c_out else nn.Sequential(
            nn.Conv2d(c_in, c_out, 1, stride, bias=False), nn.BatchNorm2d(c_out))
    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return F.relu(out + self.short(x))
```

Skip connections give gradients a direct path, enabling very deep networks.

Q257. How do you use LSTMs in PyTorch? What shapes are involved?

```
lstm = nn.LSTM(input_size=128, hidden_size=256, num_layers=2,
               batch_first=True, bidirectional=True, dropout=0.2)
out, (h_n, c_n) = lstm(x)   # x: (N, L, 128)
# out: (N, L, 2*256)   h_n, c_n: (num_layers*2, N, 256)
```

Default is `batch_first=False` (shape `(L, N, H)`). For variable-length sequences use `pack_padded_sequence(x, lengths, batch_first=True, enforce_sorted=False)` so padding isn't processed, then `pad_packed_sequence`. For classification use the final hidden state (concatenate both directions) or pooled outputs.

Q258. Implement scaled dot-product self-attention / multi-head attention.

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads, dropout=0.1):
        super().__init__()
        assert d_model % n_heads == 0
        self.h, self.dk = n_heads, d_model // n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model)
        self.out = nn.Linear(d_model, d_model)
        self.drop = nn.Dropout(dropout)
    def forward(self, x, mask=None):                    # x: (B, T, D)
        B, T, D = x.shape
        q, k, v = self.qkv(x).view(B, T, 3, self.h, self.dk).permute(2, 0, 3, 1, 4)
        att = (q @ k.transpose(-2, -1)) / math.sqrt(self.dk)   # (B, h, T, T)
        if mask is not None:
            att = att.masked_fill(mask == 0, float("-inf"))
        att = self.drop(att.softmax(-1))
        y = (att @ v).transpose(1, 2).reshape(B, T, D)
        return self.out(y)
```

In practice use `F.scaled_dot_product_attention(q, k, v, is_causal=True)`, which dispatches to FlashAttention / memory-efficient kernels. Causal mask: `torch.tril(torch.ones(T, T))`.

Q259. Implement a transformer encoder block (pre-norm).

```
class Block(nn.Module):
    def __init__(self, d, h, ff=4, p=0.1):
        super().__init__()
        self.ln1, self.ln2 = nn.LayerNorm(d), nn.LayerNorm(d)
        self.attn = MultiHeadAttention(d, h, p)
        self.mlp = nn.Sequential(nn.Linear(d, ff * d), nn.GELU(), nn.Linear(ff * d, d),
nn.Dropout(p))
    def forward(self, x, mask=None):
        x = x + self.attn(self.ln1(x), mask)
        return x + self.mlp(self.ln2(x))
```

Built-ins: `nn.TransformerEncoderLayer(d_model, nhead, batch_first=True, norm_first=True)`, `nn.MultiheadAttention` (note its `key_padding_mask` uses True = ignore).

Q260. How do you implement positional encodings and embeddings?

```
class SinusoidalPE(nn.Module):
    def __init__(self, d, max_len=5000):
        super().__init__()
        pos = torch.arange(max_len).unsqueeze(1)
        div = torch.exp(torch.arange(0, d, 2) * (-math.log(10000.0) / d))
        pe = torch.zeros(max_len, d)
        pe[:, 0::2], pe[:, 1::2] = torch.sin(pos * div), torch.cos(pos * div)
        self.register_buffer("pe", pe)
    def forward(self, x): return x + self.pe[: x.size(1)]
```

`nn.Embedding(vocab, d, padding_idx=0)` maps token IDs to vectors; learned positional embeddings use another `nn.Embedding(max_len, d)`; modern LLMs use RoPE.

Q261. How do you build a simple autoencoder or GAN training step?

Autoencoder: encoder → bottleneck → decoder; loss = MSE/BCE between input and reconstruction. **GAN** step: (1) update discriminator on real (label 1) and `fake.detach()` (label 0) with `BCEWithLogitsLoss`; (2) update generator with the discriminator's output on fake labeled as real. Separate optimizers (Adam, betas=(0.5, 0.999)); `detach` prevents generator gradients during the D step.

Q262. What is the reparameterization trick? How is it used in a VAE?

Gradients can't flow through random sampling. Instead of sampling `z ~ N(μ, σ²)` directly, sample `ε ~ N(0, 1)` and compute `z = μ + σ·ε`, which is differentiable with respect to μ and σ.

```
mu, logvar = enc(x).chunk(2, dim=-1)
z = mu + torch.exp(0.5 * logvar) * torch.randn_like(mu)
recon = dec(z)
kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp(), dim=-1).mean()
loss = F.mse_loss(recon, x, reduction="sum") / x.size(0) + kl
```

`torch.distributions` provides the same through `Normal(mu, std).rsample()` (reparameterized) vs `.sample()` (no gradient), along with `log_prob`, `entropy` and `kl_divergence`. These are also used in RL policy gradients.

Q263. What is the KV-cache in autoregressive transformers?

When generating token by token, the keys and values of earlier tokens don't change. Caching them means each new step computes Q, K and V only for the new token and attends over the cached K and V. Without a cache, each step recomputes the whole prefix (quadratic total cost); with it, the per-step cost is linear in context length. The trade-off is memory: `2 × layers × heads × head_dim × seq_len × batch × bytes`. Optimizations include MQA/GQA (shared KV heads), paged attention (vLLM), KV quantization and sliding windows.

26. Transfer Learning & Ecosystem

Q264. How do you fine-tune a pretrained torchvision model?

```
from torchvision.models import resnet50, ResNet50_Weights
weights = ResNet50_Weights.DEFAULT
model = resnet50(weights=weights)
for p in model.parameters(): p.requires_grad = False        # feature extraction
model.fc = nn.Linear(model.fc.in_features, num_classes)     # new trainable head
preprocess = weights.transforms()                            # matching preprocessing
```

Strategies: (1) **feature extraction** — freeze backbone, train head; (2) **full fine-tuning** with lower LR for backbone (discriminative LRs); (3) **gradual unfreezing**. Use the same normalization as pretraining. With small data, freeze more; with large/different data, fine-tune more.

Q265. How do you fine-tune a Hugging Face transformer with PyTorch?

```
tok = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModelForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)
batch = tok(texts, padding=True, truncation=True, max_length=256, return_tensors="pt")
out = model(**batch, labels=labels)       # out.loss, out.logits
out.loss.backward(); optimizer.step()
```

HF models are `nn.Module`s, so custom loops work; or use `Trainer`/Accelerate. For LLMs use PEFT (LoRA/QLoRA) to train small adapter matrices while freezing the base model.

Q266. What is LoRA and how does it relate to PyTorch modules?

Low-Rank Adaptation freezes a pretrained weight `W` and learns `ΔW = B A` with rank `r « d`, scaled by `α/r`. Implemented by wrapping `nn.Linear` layers (e.g., attention projections) with an extra low-rank path; only A and B require grad. Reduces trainable params by >99% and allows merging back into W for inference. QLoRA adds 4-bit quantized base weights.

Q267. What is PyTorch Lightning and why use it?

A high-level wrapper that organizes code into a `LightningModule` (`training_step`, `validation_step`, `configure_optimizers`) and a `Trainer` handling the loop, devices, DDP/FSDP, AMP, checkpointing, logging, early stopping and gradient accumulation via flags. Reduces boilerplate and errors while keeping pure PyTorch models.

Q268. Name key libraries in the PyTorch ecosystem.

torchvision (vision datasets, models, transforms, ops like NMS/RoIAlign), torchaudio, TorchText (deprecated) / HF tokenizers, timm (image models), Hugging Face transformers/datasets/peft/accelerate, PyTorch Geometric & DGL (graphs), torchmetrics, Lightning, Captum (interpretability), Optuna/Ray Tune (HPO), torchao (quantization), TorchRL, ExecuTorch, ONNX Runtime, DeepSpeed, Detectron2/MMDetection, Ultralytics YOLO.

Q269. How do you interpret/explain PyTorch models?

Captum provides Integrated Gradients, Saliency, DeepLIFT, GradCAM, SHAP approximations, occlusion. Manual Grad-CAM uses forward + backward hooks on the last conv layer to weight feature maps by pooled gradients. Attention visualization for transformers (with caution).

27. Debugging & Common Pitfalls

Q270. Your training loss is NaN. How do you debug it?

- Check inputs/labels for NaN/inf (`torch.isfinite(x).all()`) and correct normalization.
- Lower the learning rate; add warmup and gradient clipping.
- Numerical issues: `log(0)`, division by zero, softmax + log separately → use `log_softmax`, `BCEWithLogitsLoss`, add eps.
- fp16 overflow → use GradScaler or bf16.
- `torch.autograd.set_detect_anomaly(True)` to find the op producing NaN in backward (slow; debug only).
- Forward hooks to locate the first layer producing NaN.

Q271. The loss doesn't decrease. What do you check?

- Overfit a single small batch first — if it can't reach ~0 loss, there's a bug.
- Forgot `zero_grad()`, `optimizer.step()`, or optimizer created with wrong params (e.g., before replacing the head).
- Applied softmax before `CrossEntropyLoss`; wrong target dtype/shape; label/input misalignment after shuffling.
- Learning rate too high/low (LR range test); bad initialization; dead ReLUs.
- Frozen parameters accidentally; `detach()` / `.data` /numpy conversions breaking the graph.
- Inspect gradient norms per layer (vanishing/exploding).

Q272. List common PyTorch mistakes.

1. Forgetting `model.eval()` during validation/inference (or `model.train()` afterwards).
2. Not using `no_grad` / `inference_mode` in evaluation → memory blow-up.
3. Accumulating `loss` instead of `loss.item()`.
4. Softmax before `CrossEntropyLoss`; float targets for class indices.
5. Tensors on different devices; `tensor.to(device)` without reassigning.
6. Broadcasting shape bugs (`(N,)` vs `(N,1)`).
7. Using `view` on non-contiguous tensors.
8. Layers in plain Python lists (not registered).
9. Not calling `sampler.set_epoch` in DDP (same shuffle each epoch).
10. Same random augmentation seeds across DataLoader workers with NumPy RNG (older versions).
11. Scheduler stepped at the wrong frequency.
12. Loading a DDP checkpoint with `module.` prefixes into a non-DDP model.
13. Using `.data` (bypasses autograd safety) instead of `detach()` / `no_grad`.

Q273. How do you load a checkpoint saved from DDP/`torch.compile` into a plain model?

Keys are prefixed with `module.` (DDP) or `_orig_mod.` (compile). Strip them: `{k.removeprefix("module."): v for k, v in sd.items()}`, or save `model.module.state_dict()` / the original module's state dict in the first place.

Q274. What causes "Expected all tensors to be on the same device" and "expected scalar type Float but found Double"?

Device mismatch: input, model, targets, or a tensor created inside `forward` (e.g., `torch.zeros(...)` without `device=x.device`) live on different devices — create tensors with `device=x.device` or register them as buffers. Dtype mismatch: NumPy arrays are float64 by default; convert with `.float()` or `torch.from_numpy(a).float()`.

Q275. How do you detect overfitting and data leakage in a PyTorch pipeline?

Plot train vs validation curves; compare against a simple baseline; check that augmentations/normalization statistics are computed on training data only; ensure no duplicates or same-entity samples across splits; verify the validation transform is deterministic; evaluate on a held-out test set once.

28. Coding & Scenario-Based Questions

Q276. Implement linear regression from scratch using only tensors and autograd.

```
X = torch.randn(100, 3); true_w = torch.tensor([2., -1., .5])
y = X @ true_w + 0.3 + 0.1 * torch.randn(100)
w = torch.zeros(3, requires_grad=True); b = torch.zeros(1, requires_grad=True)
lr = 0.1
for _ in range(200):
    loss = ((X @ w + b - y) ** 2).mean()
    loss.backward()
    with torch.no_grad():
        w -= lr * w.grad; b -= lr * b.grad
        w.grad.zero_(); b.grad.zero_()
```

Parameter updates happen inside `no_grad` so they aren't tracked.

Q277. Implement softmax and cross-entropy in a numerically stable way.

```
def log_softmax(z, dim=-1):
    z = z - z.max(dim=dim, keepdim=True).values      # shift for stability
    return z - z.exp().sum(dim=dim, keepdim=True).log()

def cross_entropy(logits, targets):
    return -log_softmax(logits).gather(1, targets.unsqueeze(1)).mean()
```

Q278. Implement dropout and batch normalization manually.

```
def dropout(x, p, training):
    if not training or p == 0: return x
    mask = (torch.rand_like(x) > p).float()
    return x * mask / (1 - p)                  # inverted dropout keeps expectation

class MyBatchNorm1d(nn.Module):
    def __init__(self, n, eps=1e-5, momentum=0.1):
        super().__init__()
        self.gamma, self.beta = nn.Parameter(torch.ones(n)), nn.Parameter(torch.zeros(n))
        self.register_buffer("rm", torch.zeros(n)); self.register_buffer("rv", torch.ones(n))
        self.eps, self.m = eps, momentum
    def forward(self, x):
        if self.training:
            mean, var = x.mean(0), x.var(0, unbiased=False)
            with torch.no_grad():
                self.rm.lerp_(mean, self.m); self.rv.lerp_(x.var(0, unbiased=True), self.m)
        else:
            mean, var = self.rm, self.rv
        return self.gamma * (x - mean) / torch.sqrt(var + self.eps) + self.beta
```

Q279. Your GPU utilization is only 30% during training. What do you do?

Likely a data pipeline bottleneck or sync overhead. Profile; increase `num_workers`, enable `pin_memory` / `non_blocking`, `persistent_workers`; move augmentation to GPU or pre-process offline; use faster storage/formats (local SSD, WebDataset, decoded caches); remove per-step `.item()` /prints; increase batch size; use AMP and `torch.compile`; check CPU core count and contention.

Q280. Your model performs well in training/validation but poorly in production. Why?

Training-serving skew: different preprocessing (resize, normalization, tokenization) at inference; forgot `model.eval()`; different input distribution (data drift); label leakage in validation; dtype/precision differences after export/quantization; batch-dependent behavior (BatchNorm with batch size 1 in train mode). Fix by sharing preprocessing code, packaging transforms with the model, logging inputs and monitoring drift.

Q281. Design an end-to-end system to train a PyTorch image classifier and serve it via FastAPI.

1. **Data**: versioned dataset (DVC/blob storage), stratified splits, `Dataset` + augmentations.
2. **Training**: pretrained backbone fine-tuning, AMP, DDP on multiple GPUs, experiment tracking (MLflow/W&B), early stopping, checkpointing.
3. **Evaluation**: held-out test metrics, confusion matrix, calibration, error analysis.
4. **Packaging**: export state_dict / ONNX / compiled artifact; register model with version + metadata (class names, preprocessing config).
5. **Serving**: FastAPI app loads the model in lifespan, accepts `UploadFile`, validates type/size, preprocesses identically, runs inference in `inference_mode` with dynamic batching or behind Triton; returns top-k labels with probabilities.
6. **Ops**: Docker image, Kubernetes/Azure Container Apps with GPU nodes, autoscaling, health/readiness probes, auth + rate limiting, logging, Prometheus metrics, drift monitoring, canary releases, retraining pipeline.

Q282. How would you serve an LLM chatbot with streaming using FastAPI and PyTorch?

Load the model (HF transformers, bf16/4-bit) in lifespan or delegate to vLLM/TGI. Endpoint accepts chat messages (Pydantic), applies the chat template, generates tokens with `TextIteratorStreamer` in a background thread (or vLLM's async engine) and streams them via SSE (`StreamingResponse`). Add continuous batching, KV-cache management, max-token and timeout limits, auth, rate limiting, prompt-injection/content filters, and token usage logging.

Q283. Explain what happens step by step in one training iteration, from data to updated weights.

DataLoader worker reads samples → transforms → `collate_fn` builds a batch → pinned memory → async copy to GPU → forward pass builds the autograd graph and saves activations → loss computed → `backward()` traverses the graph in reverse computing VJPs, accumulating `.grad` (in DDP, gradient buckets all-reduced concurrently) → optional unscale/clip → `optimizer.step()` updates parameters using its state (momentum/Adam moments) → scheduler updates LR → gradients zeroed → graph freed.

Q284. Write a FastAPI endpoint that accepts an image and returns predictions from a torchvision model.

```
weights = ResNet50_Weights.DEFAULT
preprocess = weights.transforms()
categories = weights.meta["categories"]

@asynccontextmanager
async def lifespan(app):
    app.state.model = resnet50(weights=weights).eval()
    yield
app = FastAPI(lifespan=lifespan)

@app.post("/classify")
def classify(file: UploadFile, request: Request, top_k: int = Query(5, ge=1, le=20)):
    if file.content_type not in {"image/jpeg", "image/png"}:
        raise HTTPException(415, "Unsupported image type")
    data = file.file.read(10 * 1024 * 1024 + 1)
    if len(data) > 10 * 1024 * 1024:
        raise HTTPException(413, "File too large")
    try:
        img = Image.open(io.BytesIO(data)).convert("RGB")
    except UnidentifiedImageError:
        raise HTTPException(400, "Invalid image")
    x = preprocess(img).unsqueeze(0)
    with torch.inference_mode():
        probs = request.app.state.model(x).softmax(-1)[0]
    p, idx = probs.topk(top_k)
    return [{"label": categories[i], "prob": round(v, 4)} for v, i in zip(p.tolist(),
idx.tolist())]
```

Q285. Implement IoU and Non-Maximum Suppression (NMS).

```
def box_iou(a, b):                       # a: (N,4), b: (M,4) as x1,y1,x2,y2
    area_a = (a[:, 2] - a[:, 0]) * (a[:, 3] - a[:, 1])
    area_b = (b[:, 2] - b[:, 0]) * (b[:, 3] - b[:, 1])
    lt = torch.max(a[:, None, :2], b[None, :, :2])
    rb = torch.min(a[:, None, 2:], b[None, :, 2:])
    inter = (rb - lt).clamp(min=0).prod(-1)
    return inter / (area_a[:, None] + area_b[None, :] - inter)

def nms(boxes, scores, thr=0.5):
    order, keep = scores.argsort(descending=True), []
    while order.numel() > 0:
        i = order[0]; keep.append(i.item())
        if order.numel() == 1: break
        ious = box_iou(boxes[i:i+1], boxes[order[1:]])[0]
        order = order[1:][ious <= thr]
    return torch.tensor(keep)
```

In production use `torchvision.ops.nms` / `batched_nms` and `box_iou`.

Q286. Implement a contrastive (InfoNCE / CLIP-style) loss.

```
def clip_loss(img_emb, txt_emb, temperature=0.07):
    img = F.normalize(img_emb, dim=-1); txt = F.normalize(txt_emb, dim=-1)
    logits = img @ txt.T / temperature                # (B, B) similarity matrix
    targets = torch.arange(len(img), device=img.device)  # matching pairs on diagonal
    return (F.cross_entropy(logits, targets) + F.cross_entropy(logits.T, targets)) / 2
```

The other samples in the batch serve as negatives, so larger batches help. SimCLR applies the same idea to two augmented views of each image.

Q287. Implement text generation with temperature, top-k and top-p sampling.

```
@torch.inference_mode()
def generate(model, ids, max_new=50, temperature=0.8, top_k=50, top_p=0.9, eos_id=None):
    for _ in range(max_new):
        logits = model(ids)[:, -1, :] / max(temperature, 1e-5)
        if top_k:
            kth = torch.topk(logits, top_k).values[:, -1, None]
            logits = logits.masked_fill(logits < kth, float("-inf"))
        if top_p < 1.0:
            sorted_logits, idx = logits.sort(descending=True)
            cum = sorted_logits.softmax(-1).cumsum(-1)
            remove = cum - sorted_logits.softmax(-1) > top_p      # keep first token past p
            sorted_logits[remove] = float("-inf")
            logits = torch.full_like(logits, float("-inf")).scatter(-1, idx, sorted_logits)
        next_id = torch.multinomial(logits.softmax(-1), 1)
        ids = torch.cat([ids, next_id], dim=1)
        if eos_id is not None and (next_id == eos_id).all(): break
    return ids
```

Temperature below 1 sharpens the distribution and above 1 flattens it. Greedy decoding uses `argmax`; beam search keeps the k best sequences. Real implementations use a KV-cache instead of re-running the full prefix.

Q288. Implement k-means clustering using only PyTorch tensors.

```
def kmeans(x, k, iters=50):
    c = x[torch.randperm(len(x))[:k]].clone()
    for _ in range(iters):
        assign = torch.cdist(x, c).argmin(1)             # (N,)
        new_c = torch.zeros_like(c).index_add_(0, assign, x)
        counts = torch.bincount(assign, minlength=k).unsqueeze(1)
        new_c = torch.where(counts > 0, new_c / counts.clamp(min=1), c)   # keep empty clusters
        if torch.allclose(new_c, c): break
        c = new_c
    return c, assign
```

This tests vectorization skills: `cdist` broadcasting, `index_add_` and `bincount` instead of Python loops. It runs on GPU unchanged.

— End of Document —